

Healthcare Analytics

```
In [1]: import pandas as pd

# File path
file_path = r"C:\Users\DELL\Downloads\capstone 4th sem\healthcare project\alzheimers_disease_data.csv"

# Load the CSV file into a DataFrame
df = pd.read_csv(file_path)

# Display the first few rows of the DataFrame
print(df.head())
```

	PatientID	Age	Gender	Ethnicity	EducationLevel	BMI	Smoking	\
0	4751	73	0	0	2	22.927749	0	
1	4752	89	0	0	0	26.827681	0	
2	4753	73	0	3	1	17.795882	0	
3	4754	74	1	0	1	33.800817	1	
4	4755	89	0	0	0	20.716974	0	

	AlcoholConsumption	PhysicalActivity	DietQuality	...	MemoryComplaints	\
0	13.297218	6.327112	1.347214	...	0	
1	4.542524	7.619885	0.518767	...	0	
2	19.555085	7.844988	1.826335	...	0	
3	12.209266	8.428001	7.435604	...	0	
4	18.454356	6.310461	0.795498	...	0	

	BehavioralProblems	ADL	Confusion	Disorientation	\
0	0	1.725883	0	0	
1	0	2.592424	0	0	
2	0	7.119548	0	1	
3	1	6.481226	0	0	
4	0	0.014691	0	0	

	PersonalityChanges	DifficultyCompletingTasks	Forgetfulness	Diagnosis	\
0	0	1	0	0	
1	0	0	1	0	
2	0	1	0	0	
3	0	0	0	0	
4	1	1	0	0	

	DoctorInCharge
0	XXXConfid
1	XXXConfid
2	XXXConfid
3	XXXConfid
4	XXXConfid

[5 rows x 35 columns]

Exploratory Data Analysis (EDA)

We will first check if there's the need of Data Cleaning and Data Preprocessing by checking the shape, Null or missing values, duplicates, Outliers, Imbalance, etc then we will do proper Information extraction by doing Visualization using Matplotlib, Seboarn and Plotly

```
In [2]: pd.set_option('display.max_columns', 99)
```

```
In [3]: df.head()
```

	PatientID	Age	Gender	Ethnicity	EducationLevel	BMI	Smoking	AlcoholConsumption	PhysicalActivity	DietQuality	SleepQuality	FamilyHistoryAlzheimers	CardiovascularDisease	Diabetes	Depression	HeadInjury
0	4751	73	0	0	2	22.927749	0	13.297218	6.327112	1.347214	9.025679	0	0	1	1	0
1	4752	89	0	0	0	26.827681	0	4.542524	7.619885	0.518767	7.151293	0	0	0	0	0
2	4753	73	0	3	1	17.795882	0	19.555085	7.844988	1.826335	9.673574	1	0	0	0	0
3	4754	74	1	0	1	33.800817	1	12.209266	8.428001	7.435604	8.392554	0	0	0	0	0
4	4755	89	0	0	0	20.716974	0	18.454356	6.310461	0.795498	5.597238	0	0	0	0	0

```
In [4]: df.shape
```

Out[4]: (2149, 35)

```
In [5]: df.isna().sum()
```

PatientID	0
Age	0
Gender	0
Ethnicity	0
EducationLevel	0
BMI	0
Smoking	0
AlcoholConsumption	0
PhysicalActivity	0
DietQuality	0
SleepQuality	0
FamilyHistoryAlzheimers	0
CardiovascularDisease	0
Diabetes	0
Depression	0
HeadInjury	0
Hypertension	0
SystolicBP	0
DiastolicBP	0
CholesterolTotal	0
CholesterolLDL	0
CholesterolHDL	0
CholesterolTriglycerides	0
MMSE	0
FunctionalAssessment	0
MemoryComplaints	0
BehavioralProblems	0
ADL	0
Confusion	0
Disorientation	0
PersonalityChanges	0
DifficultyCompletingTasks	0
Forgetfulness	0
Diagnosis	0
DoctorInCharge	0
dtype: int64	

```
In [6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2149 entries, 0 to 2148
Data columns (total 35 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   PatientID                             2149 non-null   int64
1   Age                                   2149 non-null   int64
2   Gender                                2149 non-null   int64
3   Ethnicity                             2149 non-null   int64
4   EducationLevel                         2149 non-null   int64
5   BMI                                   2149 non-null   float64
6   Smoking                               2149 non-null   int64
7   AlcoholConsumption                    2149 non-null   float64
8   PhysicalActivity                       2149 non-null   float64
9   DietQuality                           2149 non-null   float64
10  SleepQuality                           2149 non-null   float64
11  FamilyHistoryAlzheimers               2149 non-null   int64
12  CardiovascularDisease                 2149 non-null   int64
13  Diabetes                              2149 non-null   int64
14  Depression                             2149 non-null   int64
15  HeadInjury                            2149 non-null   int64
16  Hypertension                           2149 non-null   int64
17  SystolicBP                            2149 non-null   int64
18  DiastolicBP                           2149 non-null   int64
19  CholesterolTotal                       2149 non-null   float64
20  CholesterolLDL                        2149 non-null   float64
21  CholesterolHDL                        2149 non-null   float64
22  CholesterolTriglycerides               2149 non-null   float64
23  MMSE                                   2149 non-null   float64
24  FunctionalAssessment                   2149 non-null   float64
25  MemoryComplaints                      2149 non-null   int64
26  BehavioralProblems                     2149 non-null   int64
27  ADL                                    2149 non-null   float64
28  Confusion                             2149 non-null   int64
29  Disorientation                         2149 non-null   int64
30  PersonalityChanges                    2149 non-null   int64
31  DifficultyCompletingTasks              2149 non-null   int64
32  Forgetfulness                          2149 non-null   int64
33  Diagnosis                              2149 non-null   int64
34  DoctorInCharge                         2149 non-null   object
dtypes: float64(12), int64(22), object(1)
memory usage: 587.7+ KB
```

Okay so no any Null Values, that saves us alot of time.

Now, lets check Duplicates

```
df['PatientID'].duplicated().sum()
```

0

By Analysing each column types and format above we can see that all of the important columns are already in Numerical format (Except DoctorInCharge), So we wont need to do any Encoding, that saves alot of time.

However, we will need to divide the Numerical columns into **Non Binary** and **Binary** so According to that we can apply Scaling later.

Now, lets investigate the ONLY text column **DoctorInCharge** and see whats going on with it.

```
df['DoctorInCharge'].duplicated().sum()
```

2148

So the DoctorInCharge have the same value for all the rows meaning its useless, and it doesn't even contribute any information to the diagnosis as well, so i will just drop it.

```
df.drop(['DoctorInCharge'], axis=1, inplace=True)
df.shape
```

(2149, 34)

```
df.dtypes
```

```
PatientID          int64
Age                int64
Gender             int64
Ethnicity          int64
EducationLevel     int64
BMI               float64
Smoking            int64
AlcoholConsumption float64
PhysicalActivity    float64
DietQuality        float64
SleepQuality       float64
FamilyHistoryAlzheimers int64
CardiovascularDisease int64
Diabetes            int64
Depression          int64
HeadInjury          int64
Hypertension        int64
SystolicBP          int64
DiastolicBP         int64
CholesterolTotal    float64
CholesterolLDL      float64
CholesterolHDL      float64
CholesterolTriglycerides float64
MMSE               float64
FunctionalAssessment float64
MemoryComplaints    int64
BehavioralProblems  int64
ADL                float64
Confusion           int64
Disorientation      int64
PersonalityChanges  int64
DifficultyCompletingTasks int64
Forgetfulness       int64
Diagnosis           int64
dtype: object
```

Alright, Now we only have Numeric columns.

Now, lets get all the Binary Numeric columns and store them into a variable for further uses.

```
binary_col = []
for col in df.columns:
    if df[col].dtype in ["int64", "float64"]:
        unique_val = df[col].nunique()
        if unique_val == 2:
            binary_col.append(col)
df[binary_col]
```

Out[11]:

	Gender	Smoking	FamilyHistory	Alzheimers	CardiovascularDisease	Diabetes	Depression	HeadInjury	Hypertension	MemoryComplaints	BehavioralProblems	Confusion	Disorientation	PersonalityChanges	DifficultyCor
0	0	0		0	0	1	1	0	0	0	0	0	0	0	0
1	0	0		0	0	0	0	0	0	0	0	0	0	0	0
2	0	0		1	0	0	0	0	0	0	0	0	1		0
3	1	1		0	0	0	0	0	0	0	1	0	0		0
4	0	0		0	0	0	0	0	0	0	0	0	0		1
...	...	...		...	...	...	...	...	...	...	...	...	...		...
2144	0	0		0	0	0	0	0	0	0	0	1	0		0
2145	0	0		0	0	0	0	0	0	0	1	0	0		0
2146	0	0		0	0	0	0	0	0	0	0	0	0		0
2147	1	0		0	1	0	0	0	0	0	0	0	0		0
2148	0	0		0	0	0	0	0	0	0	1	0	1		0

2149 rows × 16 columns

◀

▶

We will do the same to get all Non Binary columns

In [12]:

```
nonbinary_col = []
for col in df.columns:
    if df[col].dtype in ["int64", "float64"]:
        unique_val = df[col].nunique()
        if unique_val > 2:
            nonbinary_col.append(col)
df[nonbinary_col]
```

Out[12]:

	PatientID	Age	Ethnicity	EducationLevel	BMI	AlcoholConsumption	PhysicalActivity	DietQuality	SleepQuality	SystolicBP	DiastolicBP	CholesterolTotal	CholesterolLDL	CholesterolHDL	CholesterolTriglycerides
0	4751	73	0	2	22.927749	13.297218	6.327112	1.347214	9.025679	142	72	242.366840	56.150897	33.682563	162.189143
1	4752	89	0	0	26.827681	4.542524	7.619885	0.518767	7.151293	115	64	231.162595	193.407996	79.028477	294.630909
2	4753	73	3	1	17.795882	19.555085	7.844988	1.826335	9.673574	99	116	284.181858	153.322762	69.772292	83.638324
3	4754	74	0	1	33.800817	12.209266	8.428001	7.435604	8.392554	118	115	159.582240	65.366637	68.457491	277.577358
4	4755	89	0	0	20.716974	18.454356	6.310461	0.795498	5.597238	94	117	237.602184	92.869700	56.874305	291.198780
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2144	6895	61	0	1	39.121757	1.561126	4.049964	6.555306	7.535540	122	101	280.476824	94.870490	60.943092	234.520123
2145	6896	75	0	2	17.857903	18.767261	1.360667	2.904662	8.555256	152	106	186.384436	95.410700	93.649735	367.986877
2146	6897	77	0	1	15.476479	4.594670	9.886002	8.120025	5.769464	115	118	237.024558	156.267294	99.678209	294.802338
2147	6898	78	3	1	15.299911	8.674505	6.354282	1.263427	8.322874	103	96	242.197192	52.482961	81.281111	145.253746
2148	6899	72	0	2	33.289738	7.890703	6.570993	7.941404	9.878711	166	78	283.396797	92.200064	81.920043	217.396873

2149 rows × 18 columns

◀

▶

So we dont need to do any sort of Preprocessing for the Binary columns, but we do need to do some preprocessing later on the Non Binary columns.

Now, lets check Imbalance in the dataset as its important, we dont want an Imbalance in the dataset while building ML Models

Check Imbalance

In [13]:

df['Diagnosis'].value_counts()

Out[13]:

Diagnosis
0 1389
1 760
Name: count, dtype: int64

In [14]:

1389 / 760

Out[14]:

1.8276315789473685

So the Imbalance ratio is 1.82:1 indicating that there are about 1.82 times more samples in the majority class than in the minority class.

To assess the imbalance, the following general guidelines can help:

- 1:1: Perfectly balanced.
- 1:2 to 1:4: Mild imbalance.
- 1:4 to 1:10: Moderate imbalance.
- 1:10+: Severe imbalance.

So our Dataset doesn't have a Severe Imbalance but still something needed to be handled.

NOTE: I will Handle the Imabalance in the dataset after Train Test split, that way there wont be any data leakage for Training or Testing set which can happen if you Oversample before doing Splitting

Checking Outliers

lets Analyze the above Non-Binary Numeric Columns and see if there's any Outliers, we can do this simply by using the pandas describe() method first, which if you have some Domain knowledge and a Common sense you could tell if there's Outliers or not, else if you are not sure you can use Outliers Detection rules such as **3 Sigma rule**, **IQR rule** or the **Percentage Range rule**

In [15]:

df[nonbinary_col].describe()

Out[15]:

	PatientID	Age	Ethnicity	EducationLevel	BMI	AlcoholConsumption	PhysicalActivity	DietQuality	SleepQuality	SystolicBP	DiastolicBP	CholesterolTotal	CholesterolLDL	CholesterolHDL	Choles
count	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	2149.000000	
mean	5825.000000	74.908795	0.697534	1.286645	27.655697	10.039442	4.920202	4.993138	7.051081	134.264774	89.847836	225.197519	124.335944	59.463533	
std	620.507185	8.990221	0.996128	0.904527	7.217438	5.757910	2.857191	2.909055	1.763573	25.949352	17.592496	42.542233	43.366584	23.139174	
min	4751.000000	60.000000	0.000000	0.000000	15.008851	0.002003	0.003616	0.009385	4.002629	90.000000	60.000000	150.093316	50.230707	20.003434	
25%	5288.000000	67.000000	0.000000	1.000000	21.611408	5.139810	2.570626	2.458455	5.482997	112.000000	74.000000	190.252963	87.195798	39.095698	
50%	5825.000000	75.000000	0.000000	1.000000	27.823924	9.934412	4.766424	5.076087	7.115646	134.000000	91.000000	225.086430	123.342593	59.768237	
75%	6362.000000	83.000000	1.000000	2.000000	33.869778	15.157931	7.427899	7.558625	8.562521	157.000000	105.000000	262.031657	161.733733	78.939050	
max	6899.000000	90.000000	3.000000	3.000000	39.992767	19.989293	9.987429	9.998346	9.999840	179.000000	119.000000	299.993352	199.965665	99.980324	

◀

▶

So upon analyzing, We can see there doesn't seem to be any Outliers, everything seems pretty neat, so we dont need to do any further Outliers detection since this is clear enough.

From above table, We can come to a conclusion of our quick analysis:

- Average age is 74

- 75% of patients are African American.
- Average BMI is 27-28
- On average patients have less Physical activity and bad Diet quality
- Average patients SystolicBP is 134 which indicates Hypertension stage 1
- Average patients DiastolicBP is 89-90 which indicates Elavated-High Hypertension
- Average Total Cholesterol is 225 which indicates Borderline High risk for cardiovascular diseases
- Average CholesterolLDL is 124 which indicates Near Optimal/Above Optimal risk of Heart disease
- Average CholesterolHDL is 59-60 which indicates Near high lvl which is okay
- Average CholesterolTriglycerides is 228 indicates High risk of Heart attack
- Average MMSE Score is 14-15 which indicates Moderate Cognitive Impairment
- Average FunctionalAssessment Score is 5 which indicates Moderate Impairment
- Average ADL Score is 4-5 which indicates Moderate Impairment

Visualization and Deep Analysis

Check Normality

First, I want to check the Normality of each columns using **Shapiro-Wilk Test**, I will store the Columns with Non-Normal Distribution into a seperate variable so we can maybe later apply **PowerTransformer** to it

```
In [16]: from scipy.stats import shapiro
non_normal_columns = []
for column in df.columns:
    stat, p = shapiro(df[column].dropna())
    print(f'Shapiro-Wilk Test for {column}: Statistics={stat}, p={p}')
    if p < 0.05:
        non_normal_columns.append(column)
        print(f'{column} is not normally distributed (p < 0.05)')
```

Shapiro-Wilk Test for PatientID: Statistics=0.9548706515783079, p=2.9334976641191806e-25
PatientID is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Age: Statistics=0.9523114805501588, p=6.713638690721469e-26
Age is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Gender: Statistics=0.6365580360269675, p=2.005091899030555e-55
Gender is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Ethnicity: Statistics=0.7041026624829148, p=6.583909142309454e-52
Ethnicity is not normally distributed (p < 0.05)
Shapiro-Wilk Test for EducationLevel: Statistics=0.8743508124395407, p=1.5412028908226218e-38
EducationLevel is not normally distributed (p < 0.05)
Shapiro-Wilk Test for BMI: Statistics=0.9557865988815333, p=5.050001146232936e-25
BMI is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Smoking: Statistics=0.5680294087561729, p=1.8029390185737147e-58
Smoking is not normally distributed (p < 0.05)
Shapiro-Wilk Test for AlcoholConsumption: Statistics=0.9545195970652272, p=2.3874308526205607e-25
AlcoholConsumption is not normally distributed (p < 0.05)
Shapiro-Wilk Test for PhysicalActivity: Statistics=0.956680935157435, p=8.654137625068079e-25
PhysicalActivity is not normally distributed (p < 0.05)
Shapiro-Wilk Test for DietQuality: Statistics=0.9522424232989796, p=6.457166073806335e-26
DietQuality is not normally distributed (p < 0.05)
Shapiro-Wilk Test for SleepQuality: Statistics=0.9511922939888899, p=3.589806353092559e-26
SleepQuality is not normally distributed (p < 0.05)
Shapiro-Wilk Test for FamilyHistoryAlzheimers: Statistics=0.5404062622274872, p=1.3881127190496645e-59
FamilyHistoryAlzheimers is not normally distributed (p < 0.05)
Shapiro-Wilk Test for CardiovascularDisease: Statistics=0.41784994537745557, p=6.259610886963484e-64
CardiovascularDisease is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Diabetes: Statistics=0.42742923148600886, p=1.278549289411122e-63
Diabetes is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Depression: Statistics=0.4904402311640176, p=1.8381827257576053e-61
Depression is not normally distributed (p < 0.05)
Shapiro-Wilk Test for HeadInjury: Statistics=0.32730992908181367, p=1.1518798340188192e-66
HeadInjury is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Hypertension: Statistics=0.4242777872696366, p=1.0442462553529205e-63
Hypertension is not normally distributed (p < 0.05)
Shapiro-Wilk Test for SystolicBP: Statistics=0.9550825329423649, p=3.323779464075604e-25
SystolicBP is not normally distributed (p < 0.05)
Shapiro-Wilk Test for DiastolicBP: Statistics=0.9493902572125448, p=1.3400305719338252e-26
DiastolicBP is not normally distributed (p < 0.05)
Shapiro-Wilk Test for CholesterolTotal: Statistics=0.9590336004057309, p=3.7196514130717626e-24
CholesterolTotal is not normally distributed (p < 0.05)
Shapiro-Wilk Test for CholesterolLDL: Statistics=0.9536054786000656, p=1.404197086211338e-25
CholesterolLDL is not normally distributed (p < 0.05)
Shapiro-Wilk Test for CholesterolHDL: Statistics=0.9526523437342573, p=8.141950462708118e-26
CholesterolHDL is not normally distributed (p < 0.05)
Shapiro-Wilk Test for CholesterolTriglycerides: Statistics=0.9523102748862918, p=6.709072273624406e-26
CholesterolTriglycerides is not normally distributed (p < 0.05)
Shapiro-Wilk Test for MMSE: Statistics=0.9525329377348429, p=7.609064022478242e-26
MMSE is not normally distributed (p < 0.05)
Shapiro-Wilk Test for FunctionalAssessment: Statistics=0.9552213294269727, p=3.6080484074652204e-25
FunctionalAssessment is not normally distributed (p < 0.05)
Shapiro-Wilk Test for MemoryComplaints: Statistics=0.498510325005177, p=3.604561964721084e-61
MemoryComplaints is not normally distributed (p < 0.05)
Shapiro-Wilk Test for BehavioralProblems: Statistics=0.43601905185780687, p=2.4469211624181595e-63
BehavioralProblems is not normally distributed (p < 0.05)
Shapiro-Wilk Test for ADL: Statistics=0.9482232498710719, p=7.180214270346262e-27
ADL is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Confusion: Statistics=0.49552101809005644, p=2.8058039958089668e-61
Confusion is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Disorientation: Statistics=0.43796101395722475, p=2.836981379296761e-63
Disorientation is not normally distributed (p < 0.05)
Shapiro-Wilk Test for PersonalityChanges: Statistics=0.42742923148600886, p=1.278549289411122e-63
PersonalityChanges is not normally distributed (p < 0.05)
Shapiro-Wilk Test for DifficultyCompletingTasks: Statistics=0.4386050403362587, p=2.979897227983065e-63
DifficultyCompletingTasks is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Forgetfulness: Statistics=0.5766236204580888, p=4.11793742559418e-58
Forgetfulness is not normally distributed (p < 0.05)
Shapiro-Wilk Test for Diagnosis: Statistics=0.6046986534887486, p=6.773508703196818e-57
Diagnosis is not normally distributed (p < 0.05)

```
In [17]: non_normal_columns
```



```
Out[17]: ['PatientID',
        'Age',
        'Gender',
        'Ethnicity',
        'EducationLevel',
        'BMI',
        'Smoking',
        'AlcoholConsumption',
        'PhysicalActivity',
        'DietQuality',
        'SleepQuality',
        'FamilyHistoryAlzheimers',
        'CardiovascularDisease',
        'Diabetes',
        'Depression',
        'HeadInjury',
        'Hypertension',
        'SystolicBP',
        'DiastolicBP',
        'CholesterolTotal',
        'CholesterolLDL',
        'CholesterolHDL',
        'CholesterolTriglycerides',
        'MMSE',
        'FunctionalAssessment',
        'MemoryComplaints',
        'BehavioralProblems',
        'ADL',
        'Confusion',
        'Disorientation',
        'PersonalityChanges',
        'DifficultyCompletingTasks',
        'Forgetfulness',
        'Diagnosis']
```

```
In [18]: len(non_normal_columns)
```

```
Out[18]: 34
```

All of our columns are not normally distributed, we definately gonna have to apply **Power Scaling** to it during Preprocessing later.

Check Distribution

Anyways, lets just get into Plotting, we will plot a Histplot for all of the Columns and Analyze their distributions, I can simply use a For loop and plot through all the Columns at once but i want to walk through each columns, so i will be plotting one by one and im creating a new DataFrame for only Diagnosis patients, where i will do the plotting on.

```
In [19]: diagnosis_df = df[df['Diagnosis']==1]
def plot(column, dataframe=diagnosis_df):
    plt.figure(figsize=(8, 4))
    sns.histplot(dataframe[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.show()
```

I will also create a function to plot both diagnosed vs not side by side just in case we might need it

```
In [20]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# File path
file_path = r"C:\Users\DELL\Downloads\capstone 4th sem\healthcare project\alzheimers_disease_data.csv"

# Load the CSV file into a DataFrame
df = pd.read_csv(file_path)

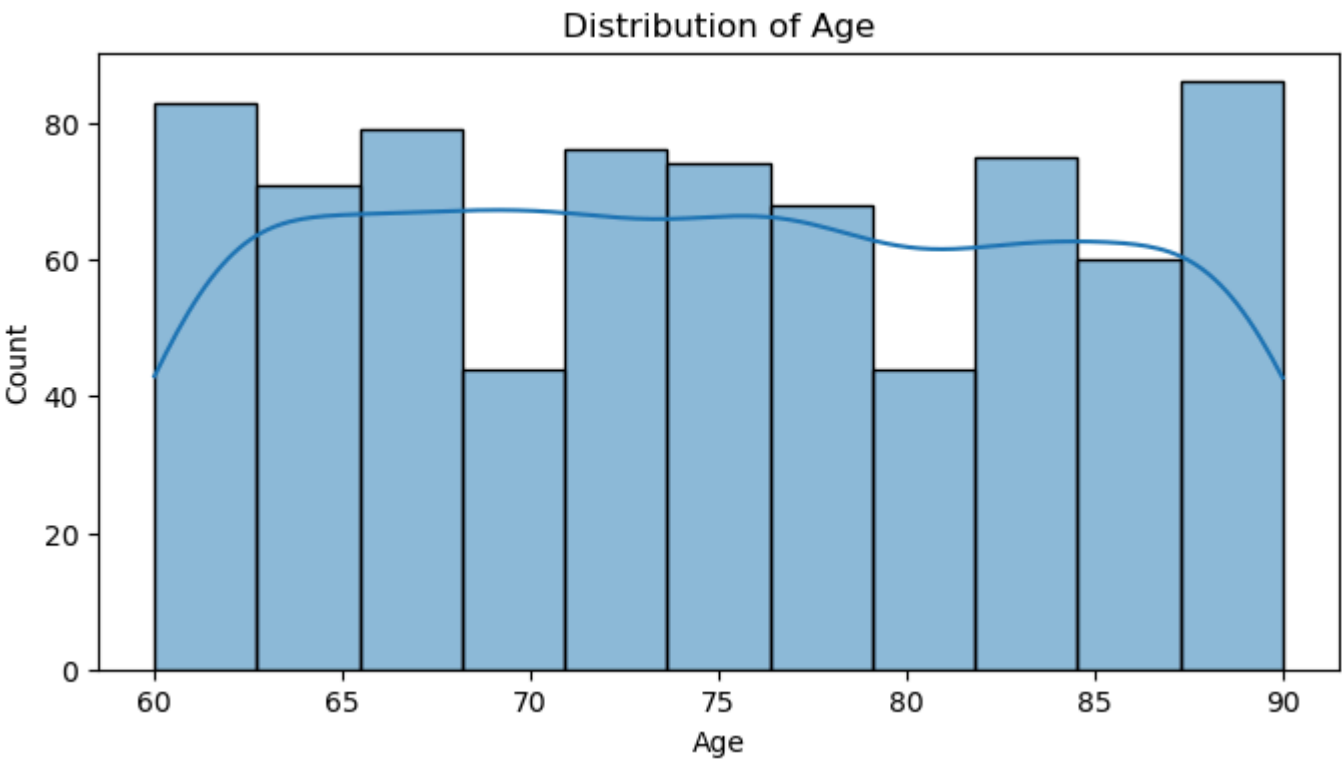
# Separate DataFrame for diagnosed patients
diagnosis_df = df[df['Diagnosis'] == 1]

# Define the plotting function with a default parameter for dataframe
def plot(column, dataframe=diagnosis_df):
    plt.figure(figsize=(8, 4))
    sns.histplot(dataframe[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.show()

# Define function to compare distributions for diagnosed and non-diagnosed
def diagnosed_vs_not(column, dataframe):
    diagnosed_df = dataframe[dataframe['Diagnosis'] == 1]
    non_diagnosed_df = dataframe[dataframe['Diagnosis'] == 0]

    fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=True)
    sns.histplot(diagnosed_df[column], kde=True, ax=axes[0])
    axes[0].set_title(f'Distribution of {column} (Diagnosed)')
    sns.histplot(non_diagnosed_df[column], kde=True, ax=axes[1])
    axes[1].set_title(f'Distribution of {column} (Non-Diagnosed)')
    plt.tight_layout()
    plt.show()
```

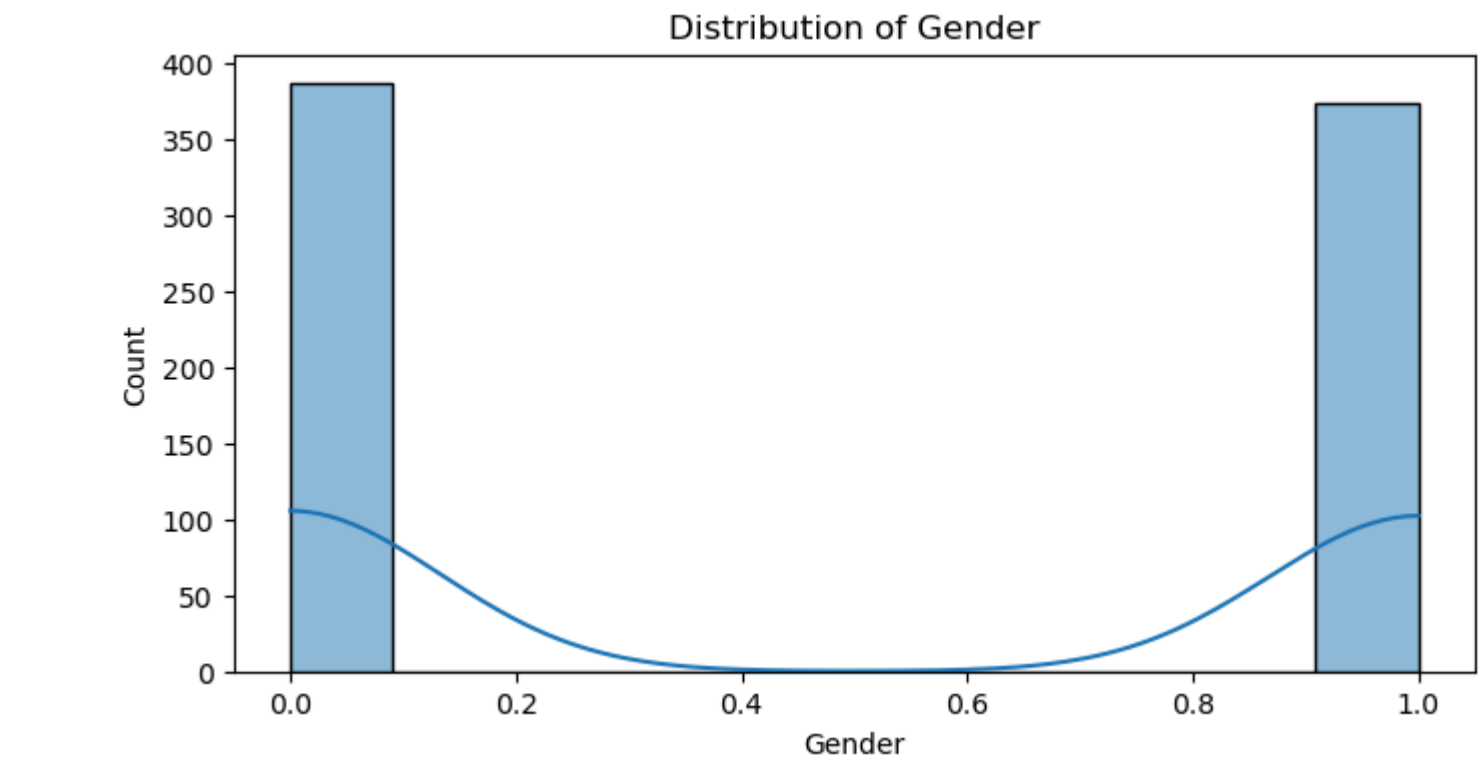
```
In [21]: plot("Age")
print(f"Average age of Alzheimer's Disease Patients is: {np.mean(df[df['Diagnosis']==1]['Age'])}")
```



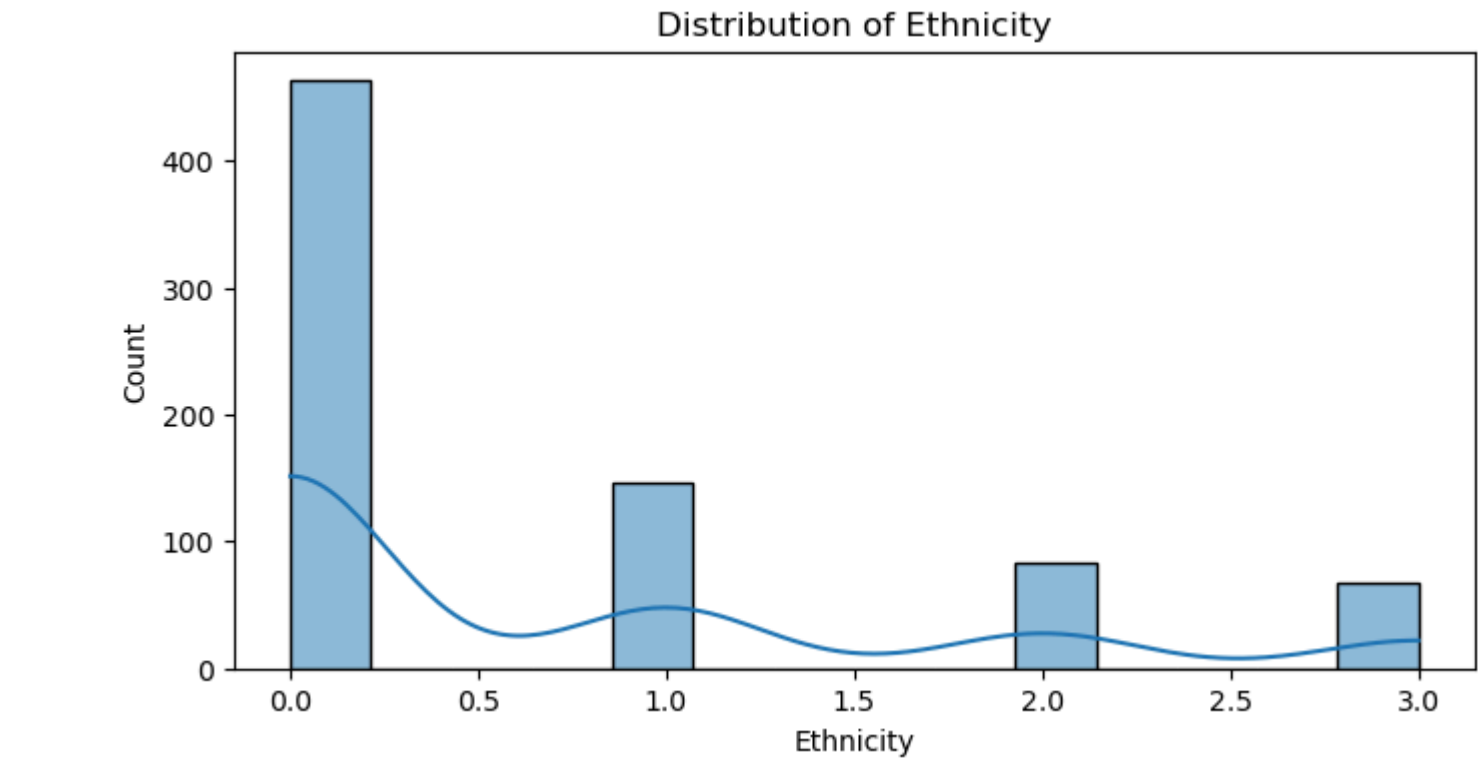
Average age of Alzheimer's Disease Patients is: 74.84210526315789

Well the Age column have a Uniform Distribution, we already saw above while checking the Normality of each columns that non of the columns have a Normal distirbution, so im assuming most of them are Uniform distribution, and the thing about Uniform distribution is its not very informative, but anyways, Average age of Patience who have Parkinson's Disease is 74 years old, which make sense because Alzheimer's Disease usually develops in middle or late life, with the average age of diagnosis being 70-75 years old and by age 85, the risk reaches nearly 50%.

```
In [22]: plot("Gender")
```



In [23]: `plot("Ethnicity")`



Okay, so most Patients with Alzheimer's Disease are likely to be Caucasian, well it could also be maybe because in our Dataset there are more Caucasian as Caucasians are the most studied group in Alzheimer's research, lets check..

In [24]: `df['Ethnicity'].value_counts()`

Out[24]:

Ethnicity	count
0	1278
1	454
3	211
2	206

Name: count, dtype: int64

Yh, i was right, hmm, so lets do one thing, we will create seperate Dataframe for each Ethnicity and calculate the Percentage of each of them who are Diagnosed with the disease and see how many percentage have the disease

```
In [25]: import pandas as pd
import plotly.express as px

# Create separate DataFrames for each ethnicity
cau = df[df['Ethnicity'] == 0]
afr_am = df[df['Ethnicity'] == 1]
asian = df[df['Ethnicity'] == 2]
other = df[df['Ethnicity'] == 3]

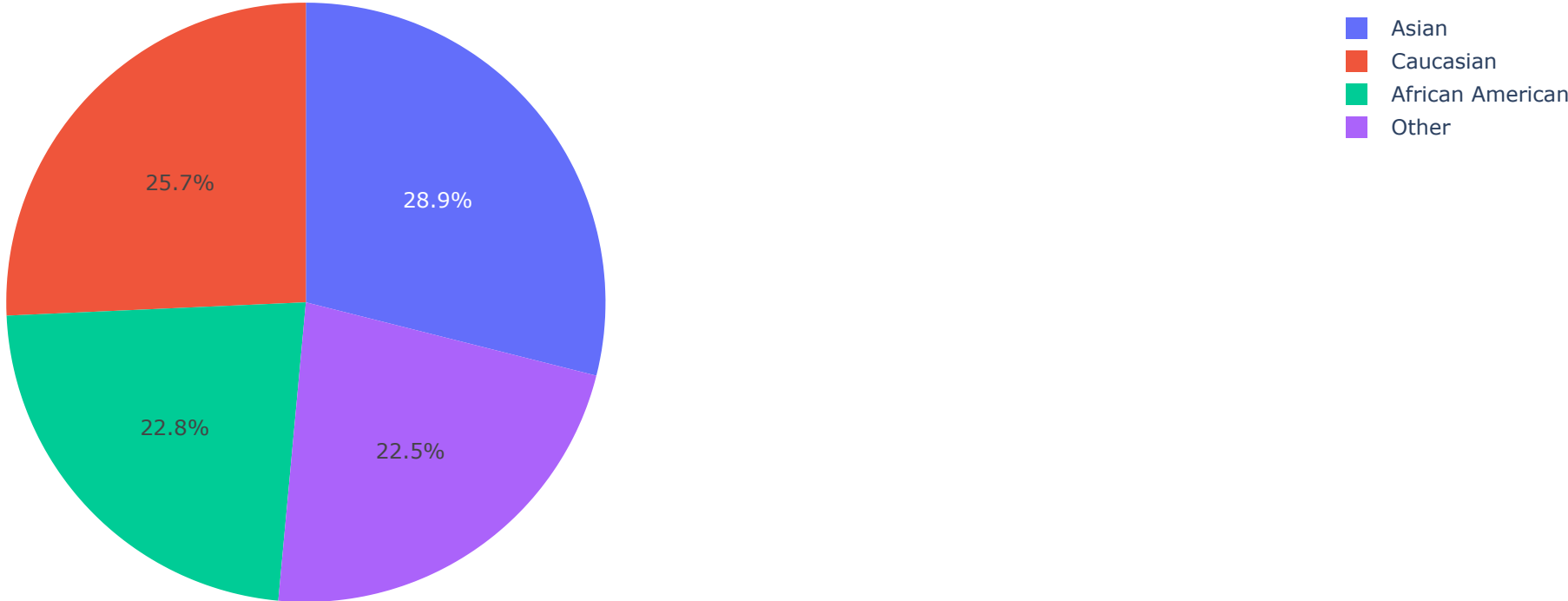
# Calculate the percentage of patients diagnosed with Alzheimer's Disease in each ethnicity group
def calculate_percentage(df):
    diagnosed = df[df['Diagnosis'] == 1]
    percentage = (len(diagnosed) / len(df)) * 100
    return percentage

cau_percentage = calculate_percentage(cau)
afr_am_percentage = calculate_percentage(afr_am)
asian_percentage = calculate_percentage(asian)
other_percentage = calculate_percentage(other)

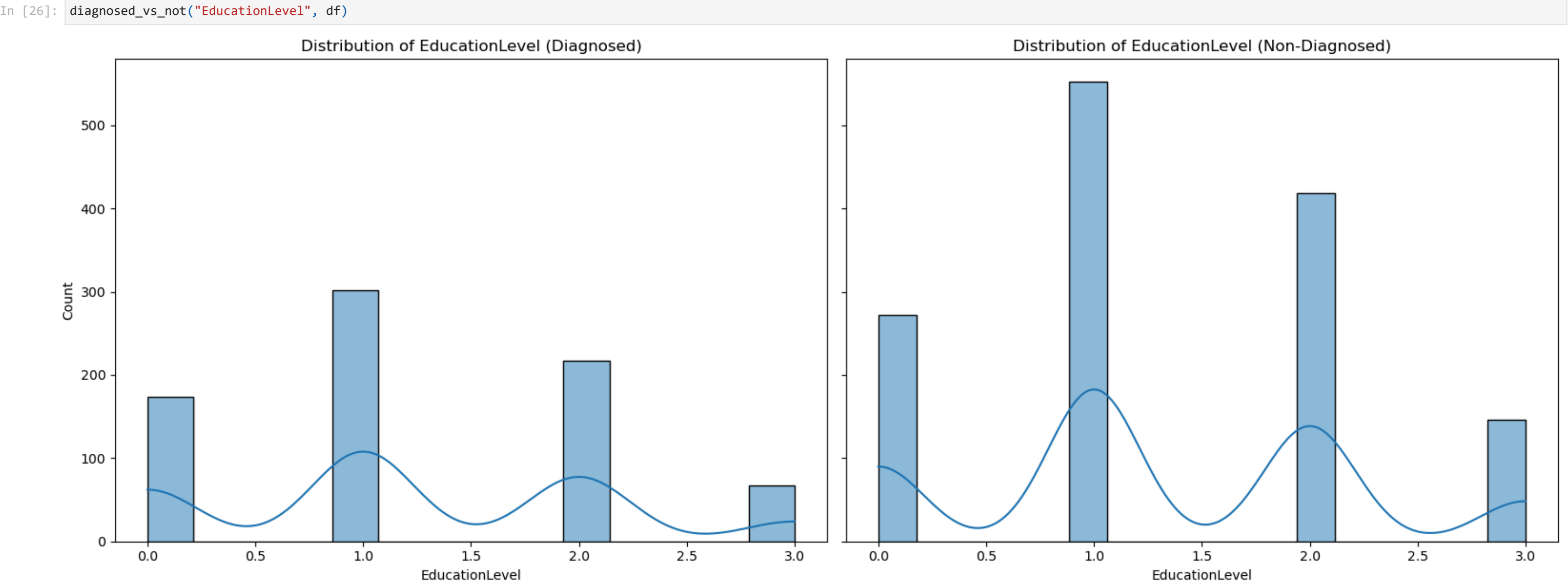
# Create a DataFrame for plotting
data = {
    'Ethnicity': ['Caucasian', 'African American', 'Asian', 'Other'],
    'Percentage Diagnosed': [cau_percentage, afr_am_percentage, asian_percentage, other_percentage]
}
plot_df = pd.DataFrame(data)

# Plot a pie chart using Plotly
fig = px.pie(plot_df, names='Ethnicity', values='Percentage Diagnosed', title="Percentage of Alzheimer's Disease Diagnoses by Ethnicity")
fig.show()
```

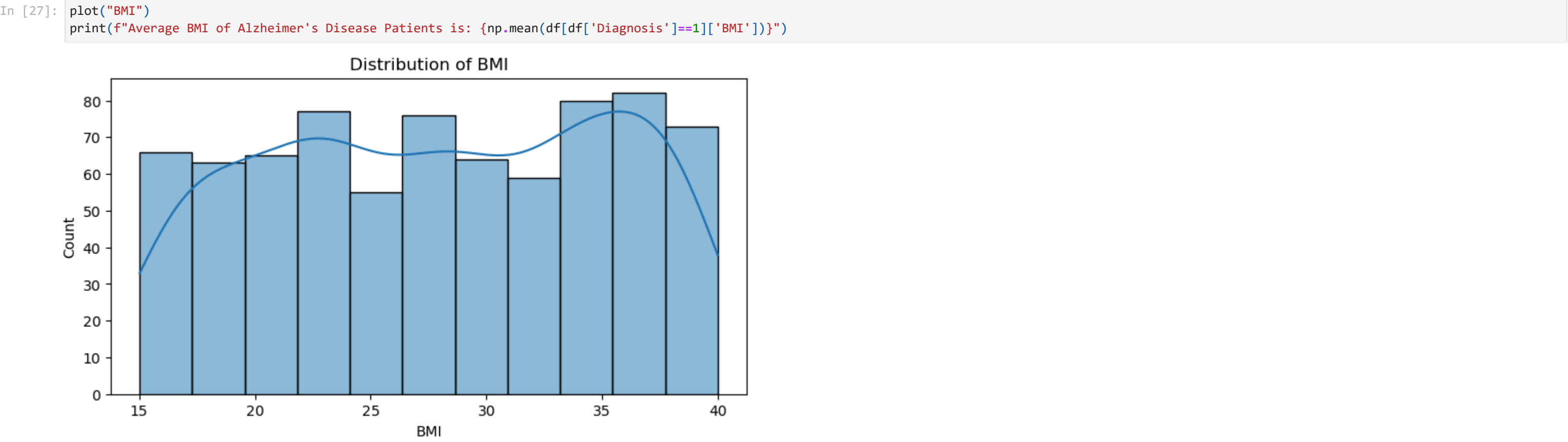
Percentage of Alzheimer's Disease Diagnoses by Ethnicity



Welp, we can see a different story from the above pie chart, but its more of equally distributed



Lifestyle Factors Visualization



Average BMI of Alzheimer's Disease Patients is: 27.912670487663743

Okay, so BMI stands for Body Mass Index. It's a measure that uses a person's height and weight to estimate whether they are underweight, normal weight, overweight, or obese.

It categorizes individuals into different weight status categories:

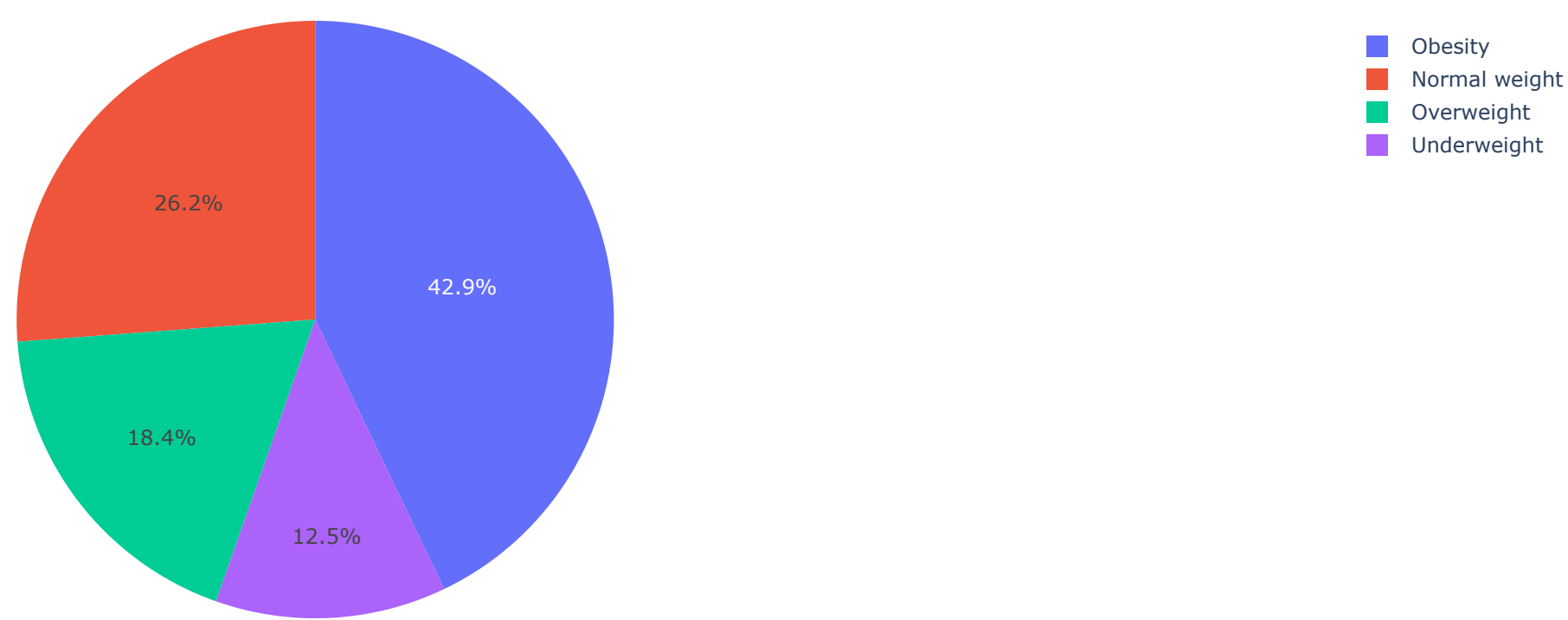
- Underweight: BMI less than 18.5
- Normal weight: BMI 18.5 to 24.9
- Overweight: BMI 25 to 29.9
- Obesity: BMI 30 or higher

Although our BMI column have kind of a Uniform Distribution, we can still see the Average BMI of Alzheimer's Disease Patients is 27, But lets Categorize our BMI column into 4 categories for Patients with Alzheimer's Disease and plot a Pie Chart

In [28]:

```
def categorize_bmi(bmi):  
    if bmi < 18.5:  
        return 'Underweight'  
    elif 18.5 <= bmi <= 24.9:  
        return 'Normal weight'  
    elif 25 <= bmi <= 29.9:  
        return 'Overweight'  
    else:  
        return 'Obesity'  
  
# Apply the function to create a new column  
df['BMI_Category'] = diagnosis_df['BMI'].apply(categorize_bmi)  
  
# Count the occurrences of each category  
bmi_counts = df['BMI_Category'].value_counts().reset_index()  
bmi_counts.columns = ['BMI_Category', 'Count']  
  
# Plot the pie chart using Plotly  
fig = px.pie(bmi_counts, values='Count', names='BMI_Category', title='BMI Categories Distribution')  
fig.show()
```

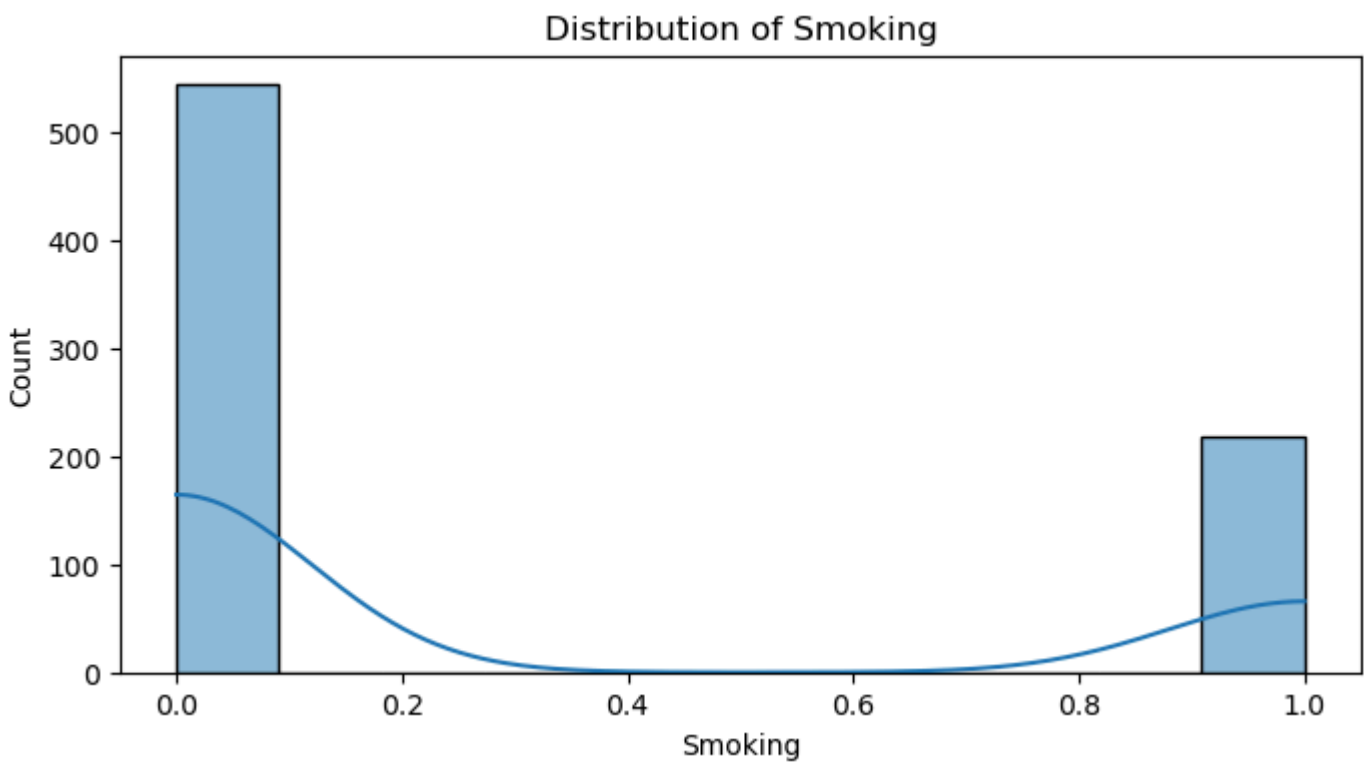

BMI Categories Distribution



```
In [29]: df.drop(['BMI_Category'], axis=1, inplace=True)
```

According to the Obesity Action Coalition, adults with a BMI of 25.0–29.9 in mid-life (ages 45–55) have a higher chance of developing Alzheimer’s disease or dementia later in life (ages 65 and older). Adults with a BMI of more than 30.0 in mid-life have an even higher risk of developing Alzheimer’s disease and vascular dementia later in life, and we can see on average the Patients who are Diagnosed are between the 25-29 range with 40% of them being in the Obese category

```
In [30]: plot("Smoking")
```



Smoking is a risk factor for Alzheimer's disease (AD) and other types of dementia. The World Health Organization (WHO) estimates that smoking is responsible for 14% of AD cases worldwide. Smoking can increase the risk of AD in a number of ways.

We can see that more people dont smoke, but again this could be due to Imbalance in the dataset, lets analyze further

```
In [31]: df['Smoking'].value_counts()
```

```
Out[31]: Smoking
0      1529
1       620
Name: count, dtype: int64
```

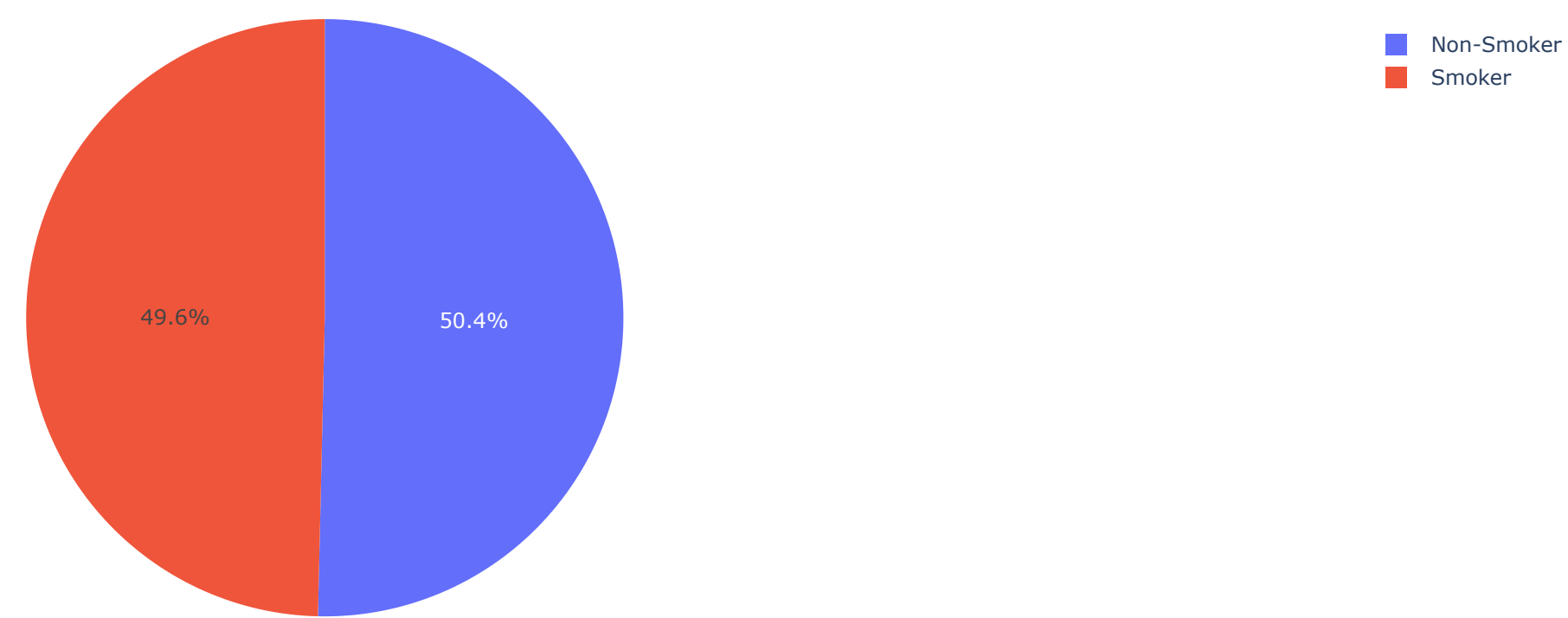
```
In [32]: # Create separate DataFrames for smokers and non-smokers
smoke = df[df['Smoking'] == 1]
no_smoke = df[df['Smoking'] == 0]

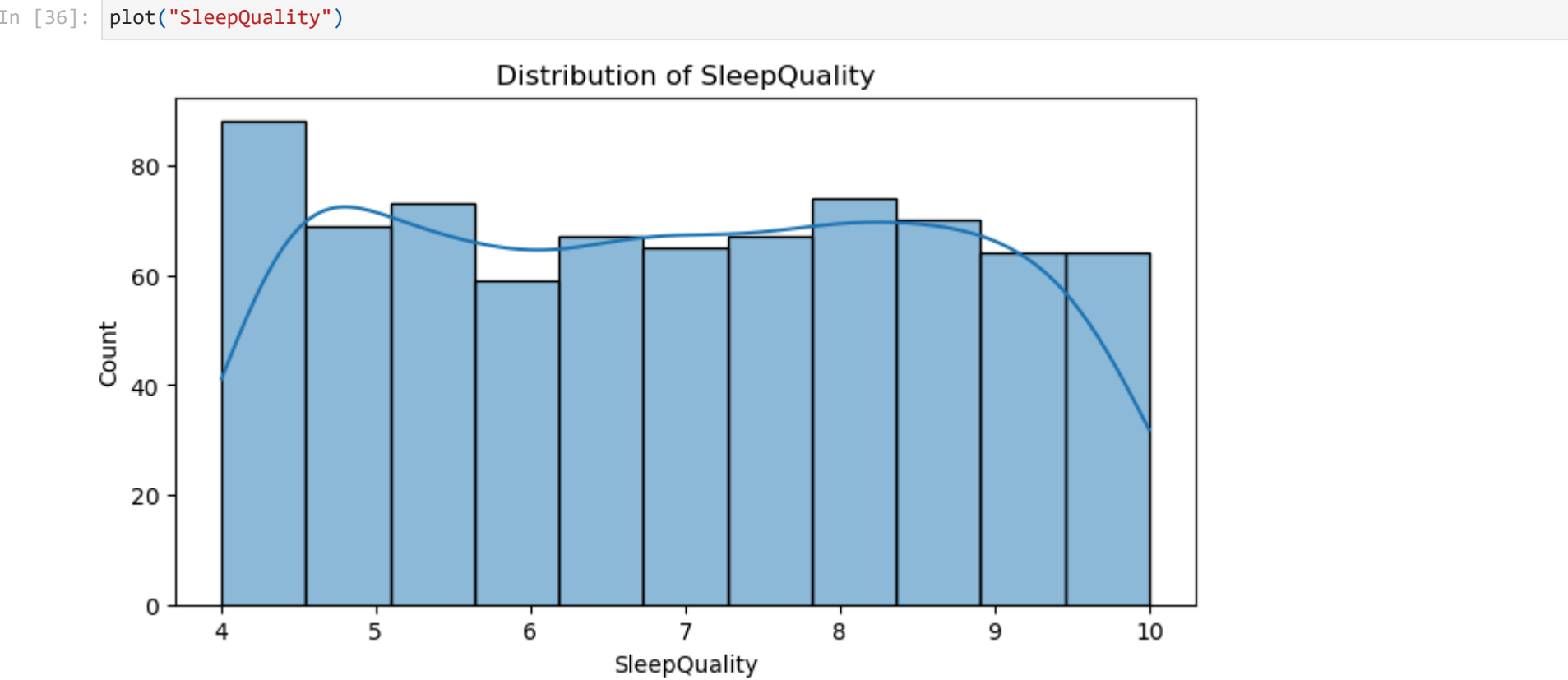
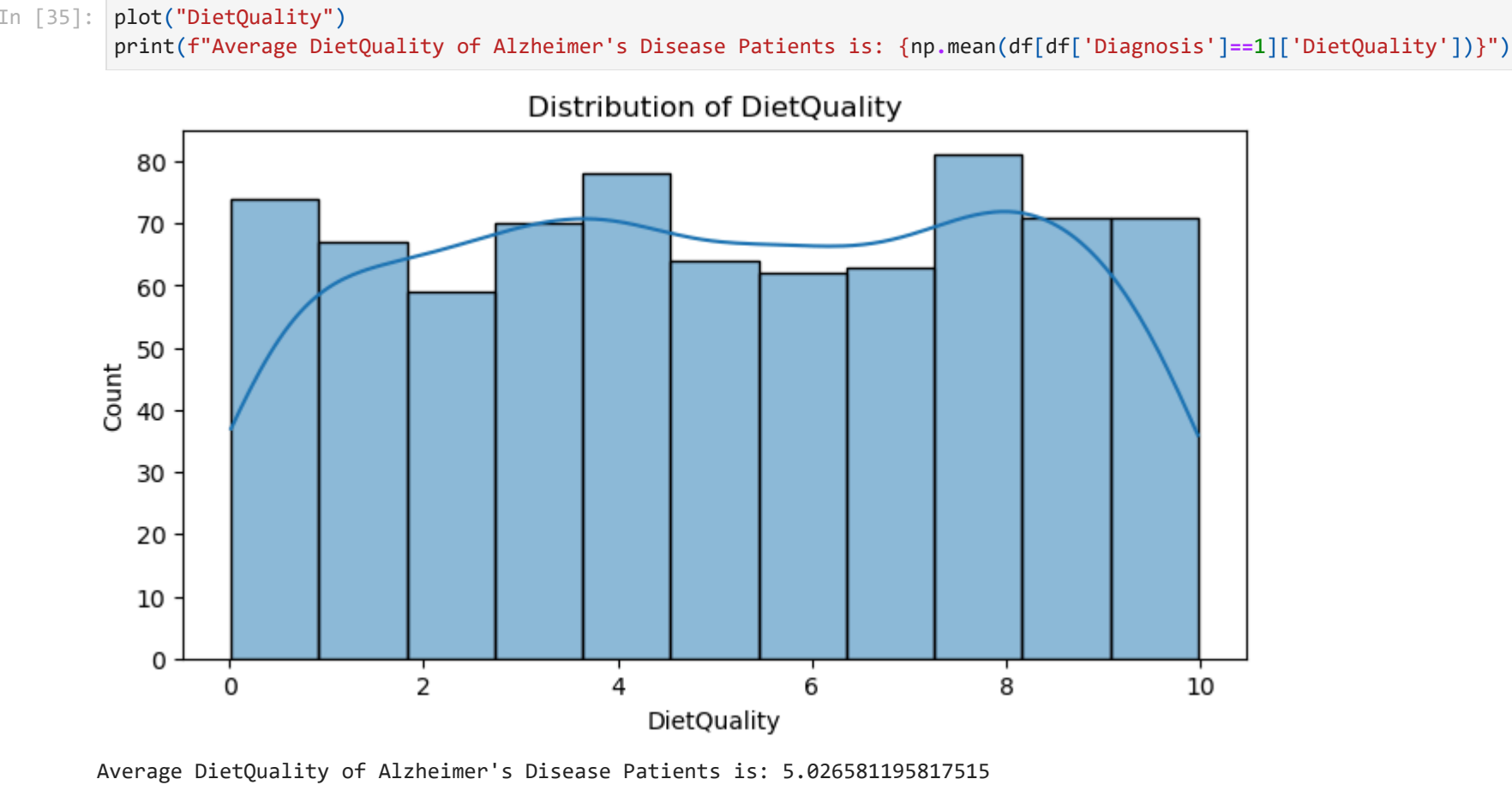
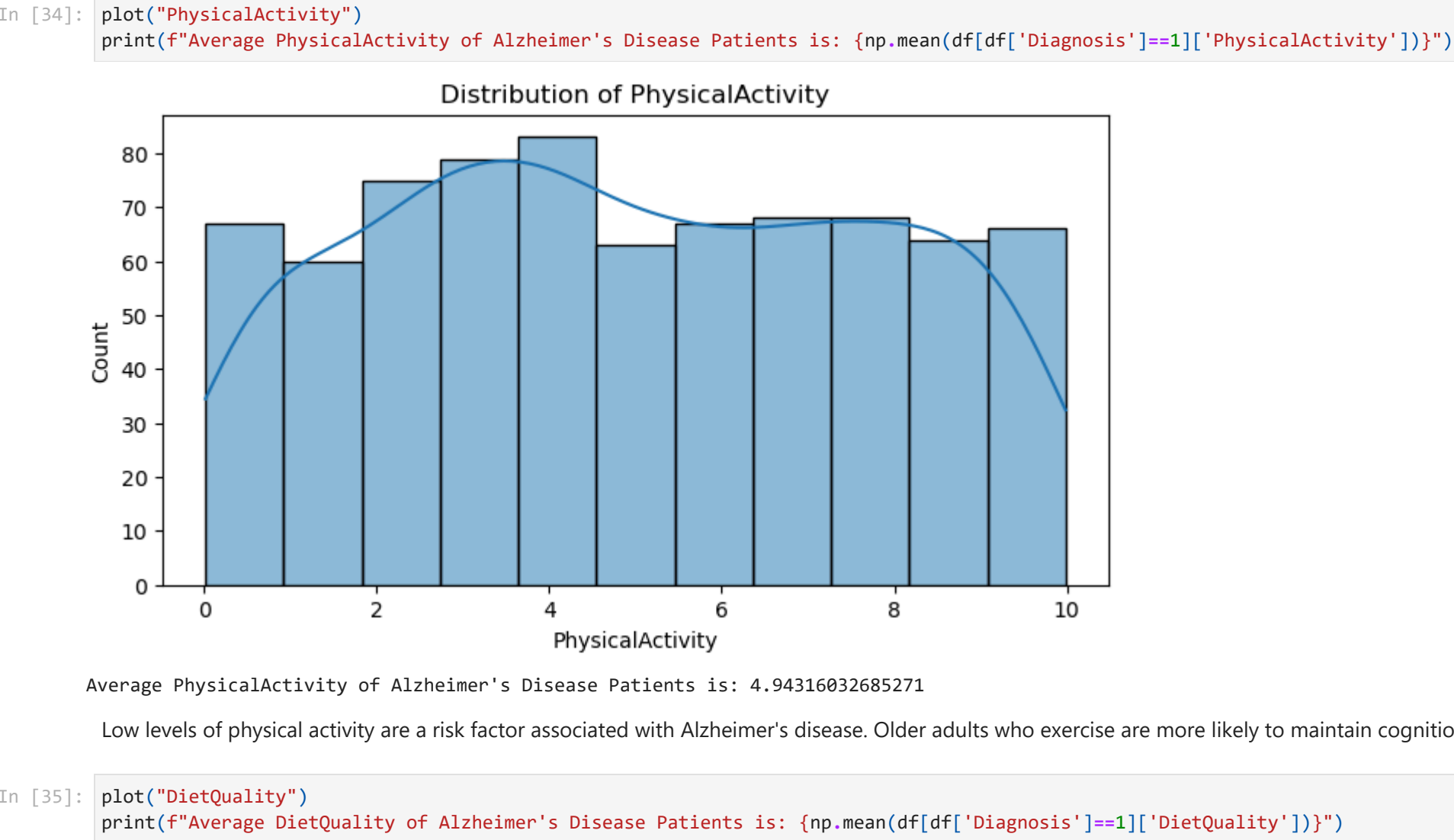
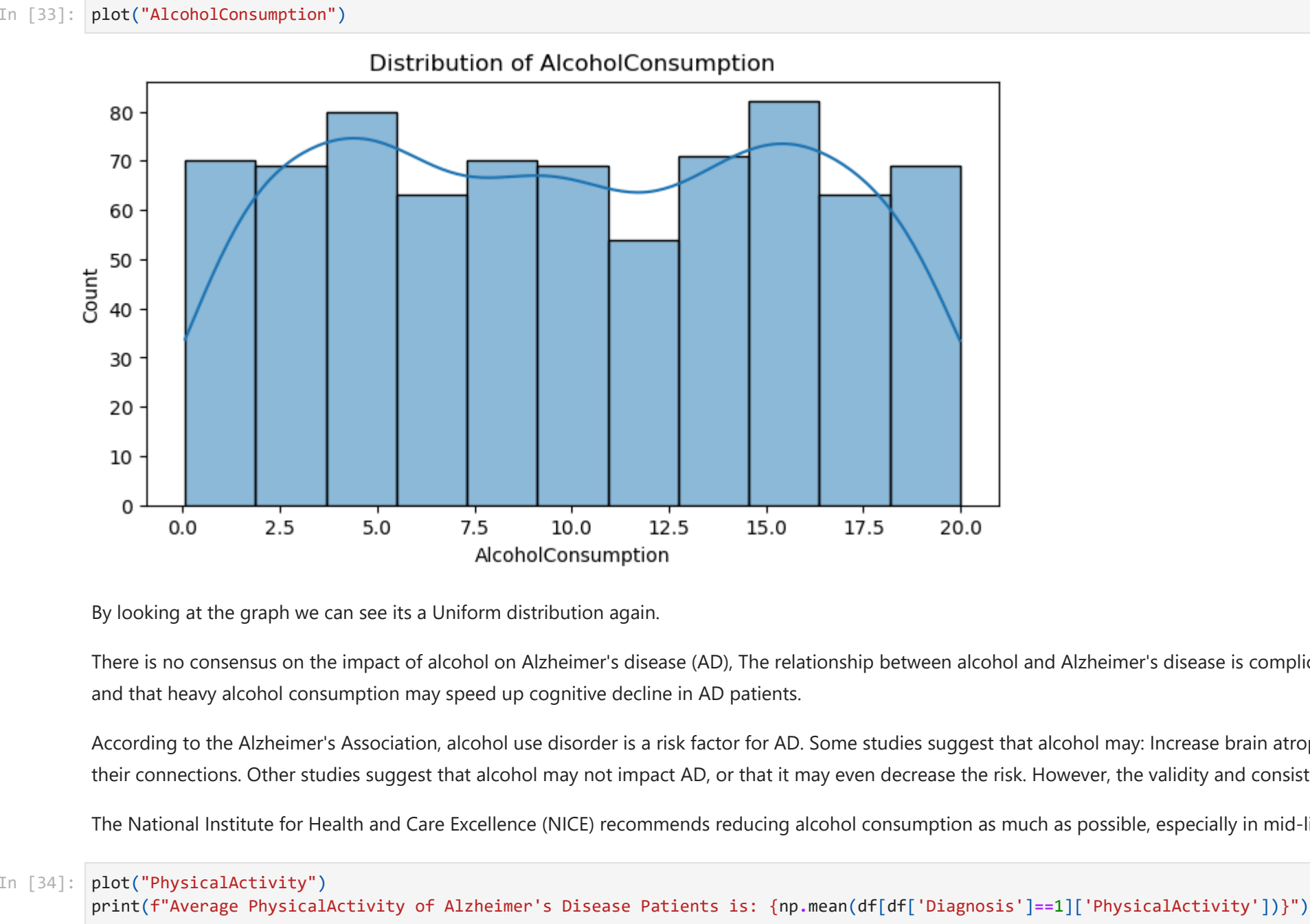
# Calculate the percentage of patients diagnosed with Alzheimer's Disease for smokers and non-smokers
smoke_per = calculate_percentage(smoke)
nosmoke_per = calculate_percentage(no_smoke)

# Create a DataFrame for plotting
data = {
    'Smoking': ['Non-Smoker', 'Smoker'],
    'Percentage Diagnosed': [nosmoke_per, smoke_per]
}
plot_df = pd.DataFrame(data)

# Plot a pie chart using Plotly
fig = px.pie(plot_df, names='Smoking', values='Percentage Diagnosed', title='Percentage of Alzheimer\'s Disease Diagnoses by Smoking Status')
fig.show()
```

Percentage of Alzheimer's Disease Diagnoses by Smoking Status





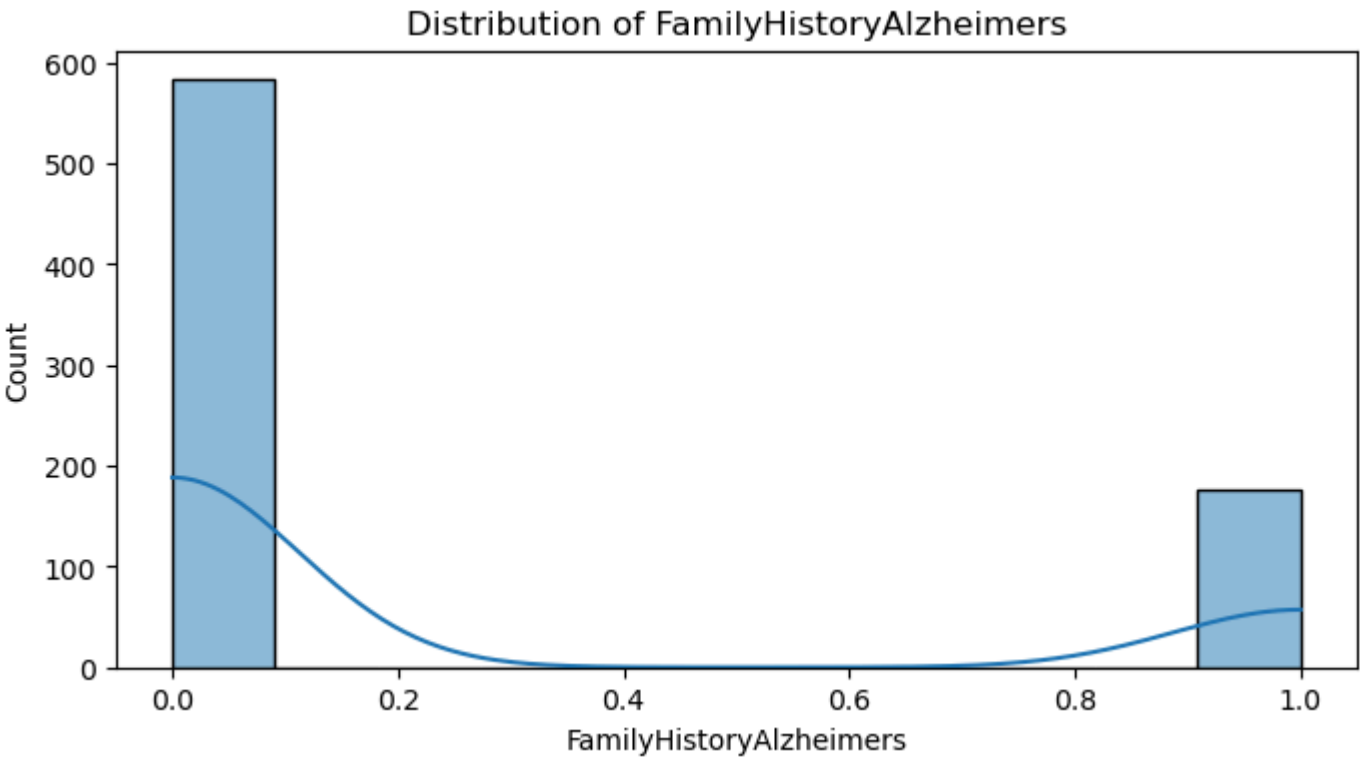
Poor sleep quality and sleep disturbances are associated with impaired cognitive function. In Alzheimer's patients, inadequate sleep can exacerbate memory problems and other cognitive deficits. Sleep disturbances are common in AD and may contribute to the progression of the disease.

Poor sleep can worsen behavioral and psychological symptoms of dementia (BPSD), such as agitation, depression, and anxiety, leading to increased caregiver burden.

Improving sleep quality in Alzheimer's Disease patients requires a comprehensive approach that includes lifestyle modifications, environmental adjustments, and, when necessary, medical interventions. Good sleep hygiene practices and a supportive sleep environment can significantly enhance the quality of life for patients

Patient's Medical History Visualization

```
In [37]: plot("FamilyHistoryAlzheimers")
```



Most of the Patients dont have any Family history of the Disease, Family history is not necessary for an individual to develop Alzheimer's. However, research shows that those who have a parent or sibling living with Alzheimer's are more likely to develop the disease than those who do not have a first-degree relative with Alzheimer's.

Lets do one thing, we will analyse the percentage of Patients having diagnosed with Alzheimer who has a family history of the disease

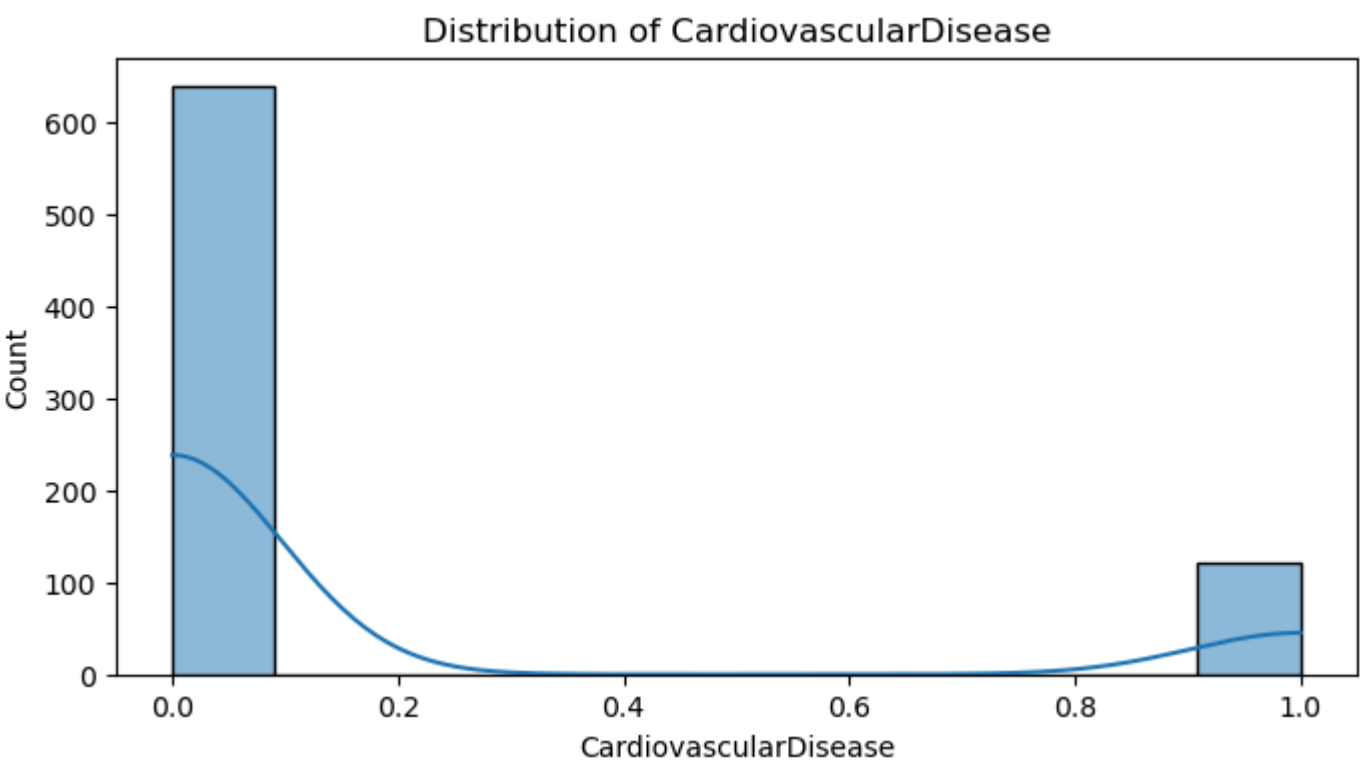
```
In [38]: def check_percent(col, df=df):
col_name = df[df[col]==1]
print(len(col_name[col_name['Diagnosis']==1]) / len(col_name) * 100)
```

```
In [39]: check_percent("FamilyHistoryAlzheimers")
```

32.656826568265686

32% Percent of patients who have a family history of the disease have also developed it

```
In [40]: plot("CardiovascularDisease")
```



Most patients dont have any Cardiovascular Disease.

Cardiovascular disease (CVD) is a term for a range of conditions that affect the heart and blood vessels. Common types include coronary artery disease (which can lead to heart attacks), hypertension (high blood pressure), heart failure, and stroke. Essentially, it's any disease that affects the heart or blood vessels and can lead to serious health problems if not managed properly.

Cardiovascular disease is related to Alzheimer's disease because they share several risk factors and underlying mechanisms. Poor cardiovascular health, such as high blood pressure, high cholesterol, obesity, diabetes, and smoking, can increase the risk of developing Alzheimer's. The connection is primarily due to the impact that cardiovascular health has on brain health.

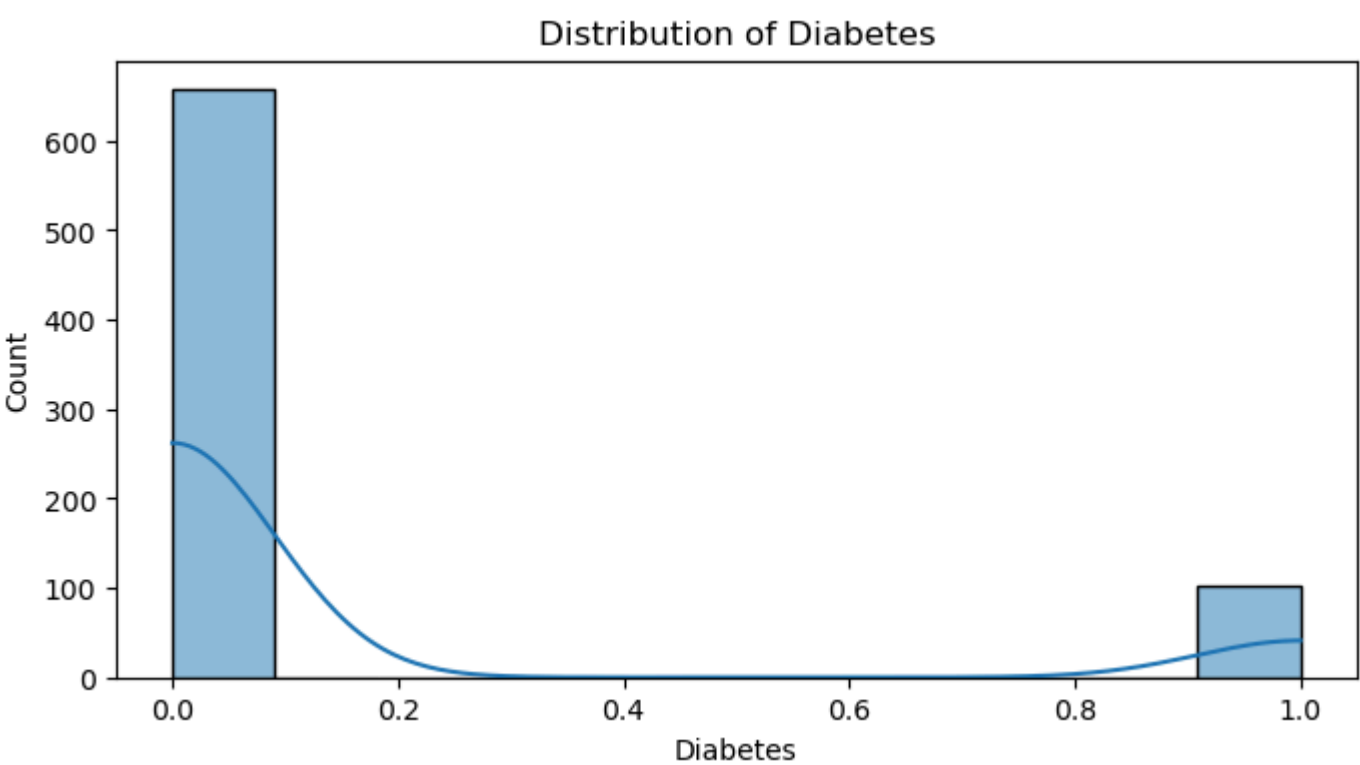
So lets look into the Patients who have Cardiovascular Disease and see how many of them are actually diagnosed with Alzheimer's as well

```
In [41]: check_percent("CardiovascularDisease")
```

39.03225806451613

almost 40% of the patients with Cardiovascular Disease have developed Alzheimer's

```
In [42]: plot("Diabetes")
```



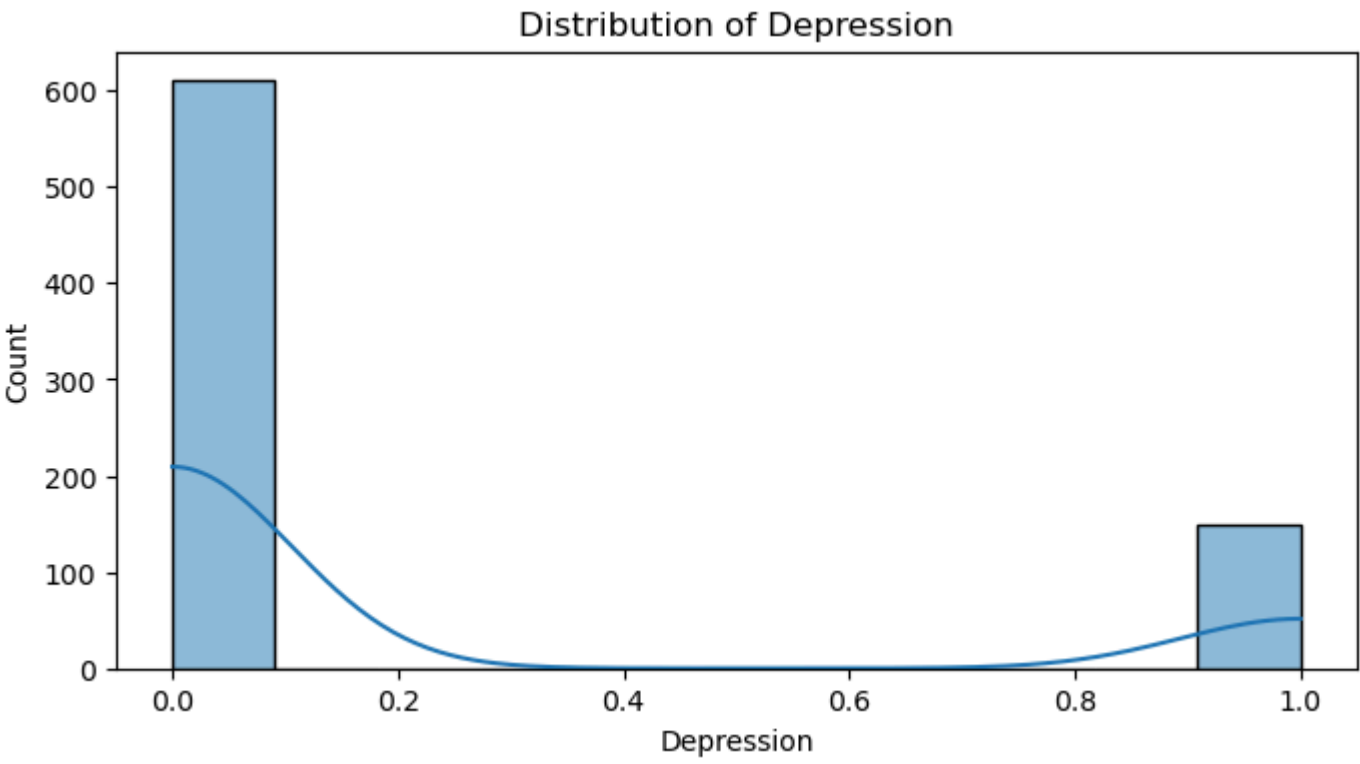
So its same here, most diagnosed patients dont have Diabete.

Diabetes is a condition where the body either doesn't produce enough insulin or can't use insulin properly, leading to high blood sugar levels. This can cause damage to the body's organs, including the heart and blood vessels, increasing the risk of cardiovascular diseases. Poor blood sugar control can also affect the brain, leading to an increased risk of developing Alzheimer's disease and other forms of dementia, although i dont think Diabeets contribute as much, probably like 30%

In [43]: `check_percent("Diabetes")`

31.790123456790127

In [44]: `plot("Depression")`

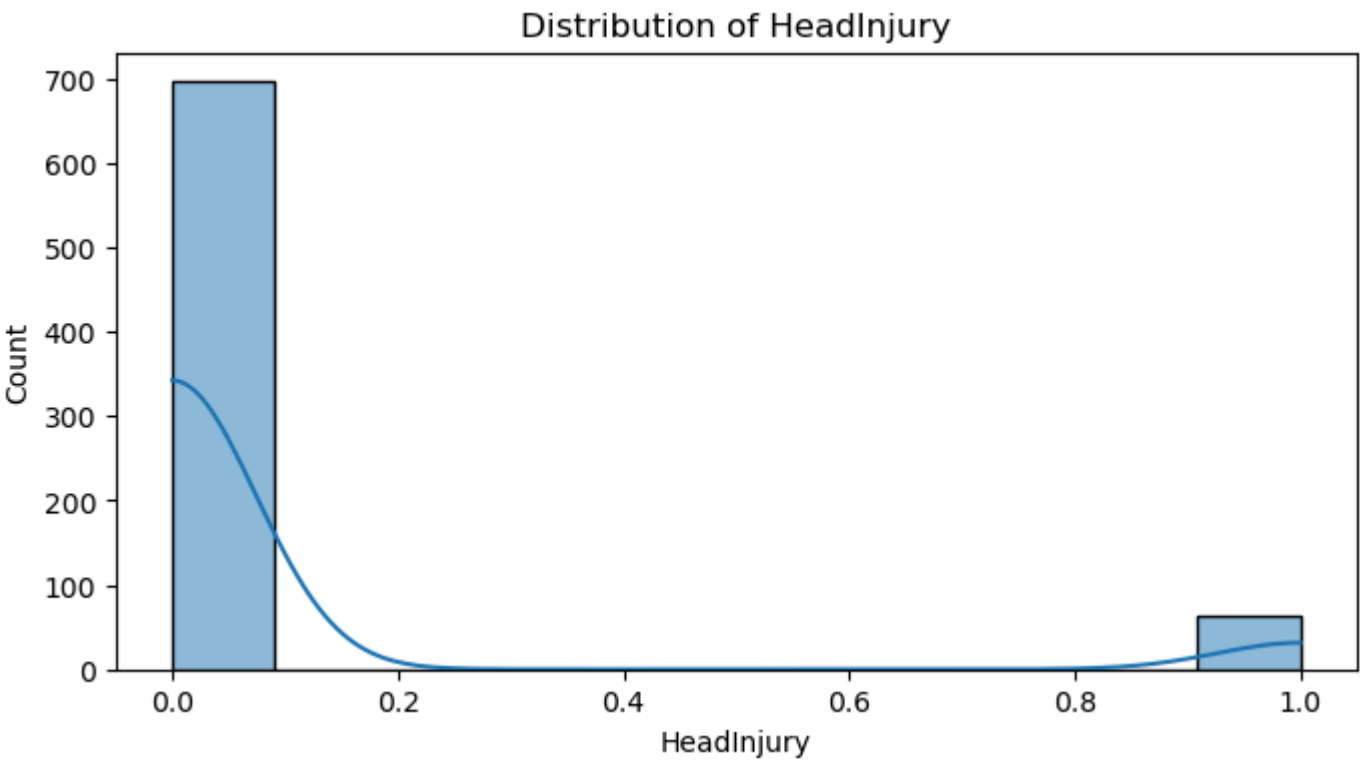


Depression is linked to an increased risk of developing Alzheimer's disease. While it's difficult to quantify exactly how much depression contributes to Alzheimer's on a specific scale, it is considered a significant risk factor. Depression can affect brain function and structure, potentially leading to changes that increase the likelihood of cognitive decline and dementia. Studies suggest that chronic depression, especially later in life, can double the risk of developing Alzheimer's disease

In [45]: `check_percent("Depression")`

34.80278422273782

In [46]: `plot("HeadInjury")`

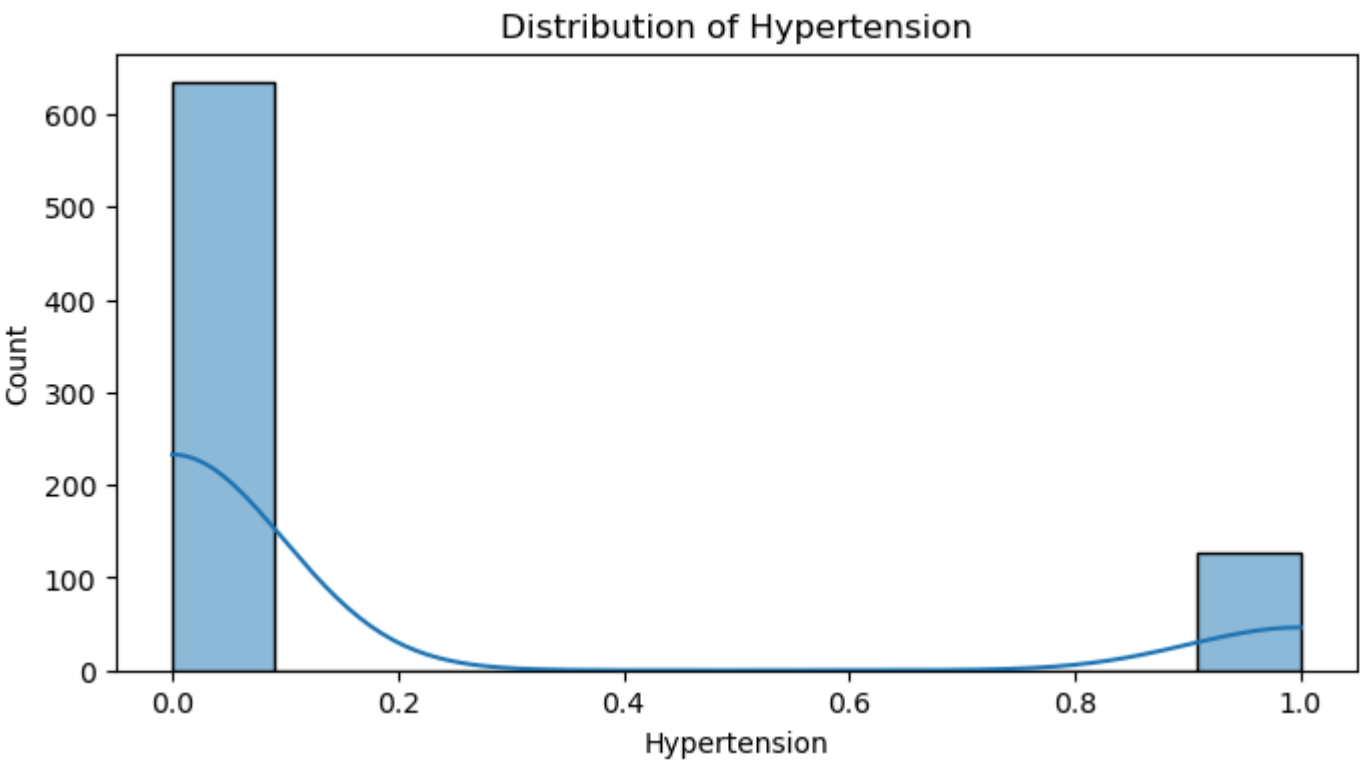


People who have had a head injury are more likely to develop Alzheimer's disease. Head injuries, especially severe or repeated ones, can damage brain cells and lead to long-term changes in brain function. This damage can increase the risk of cognitive decline and dementia, including Alzheimer's disease. For example, traumatic brain injury (TBI) has been linked to a higher risk of Alzheimer's and other forms of dementia later in life.

In [47]: `check_percent("HeadInjury")`

32.1608040201005

In [48]: `plot("Hypertension")`



Hypertension is also a significant risk factor for Alzheimer's disease. High blood pressure can lead to damage in the blood vessels of the brain, reducing blood flow and causing brain cell damage. This can contribute to cognitive decline and increase the risk of Alzheimer's and other forms of dementia.

In [49]: `check_percent("Hypertension")`

39.375

In []:

Clinical Measurements Visualization

In [50]: `import plotly.express as px`

```
clinic_col = ["SystolicBP", "DiastolicBP", "CholesterolTotal", "CholesterolLDL", "CholesterolHDL", "CholesterolTriglycerides"]
```

```
for col in clinic_col:
    fig = px.histogram(df, x=col, title=f'Histogram of {col}',
                      color_discrete_sequence=['#FF073A'], # Darker blue color
                      opacity=0.8)
```

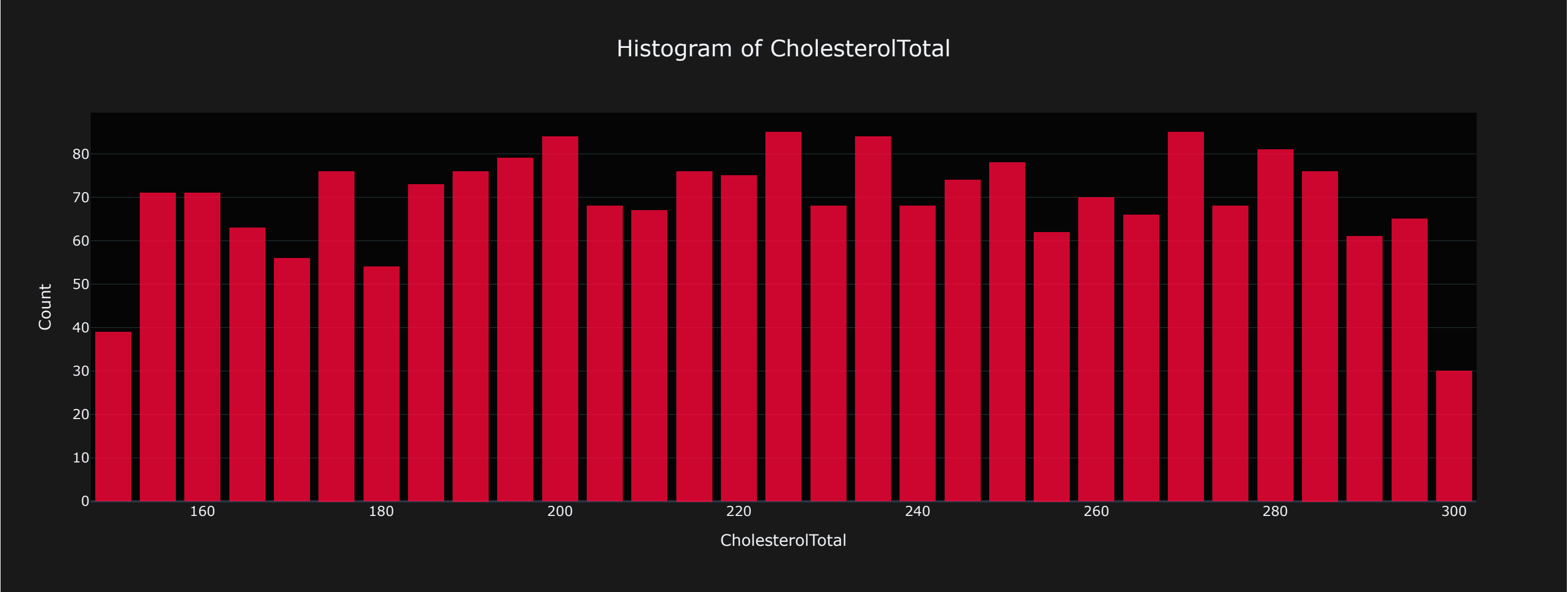
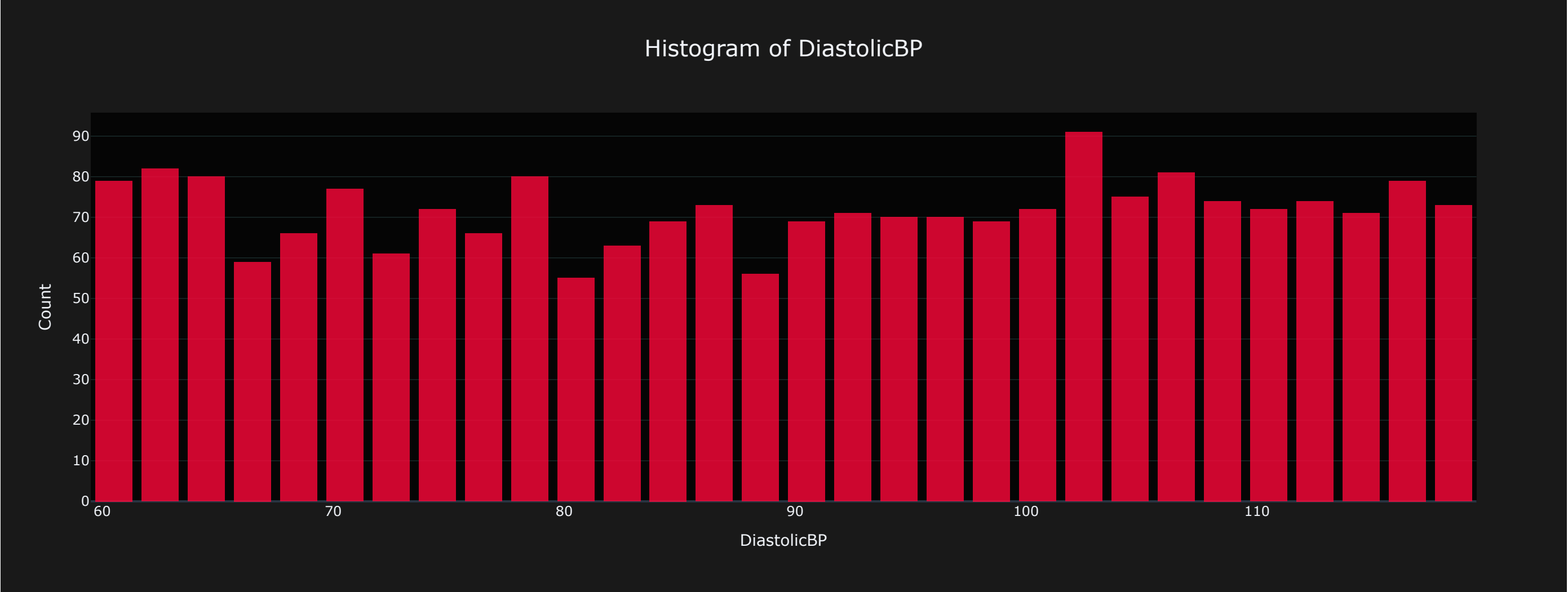
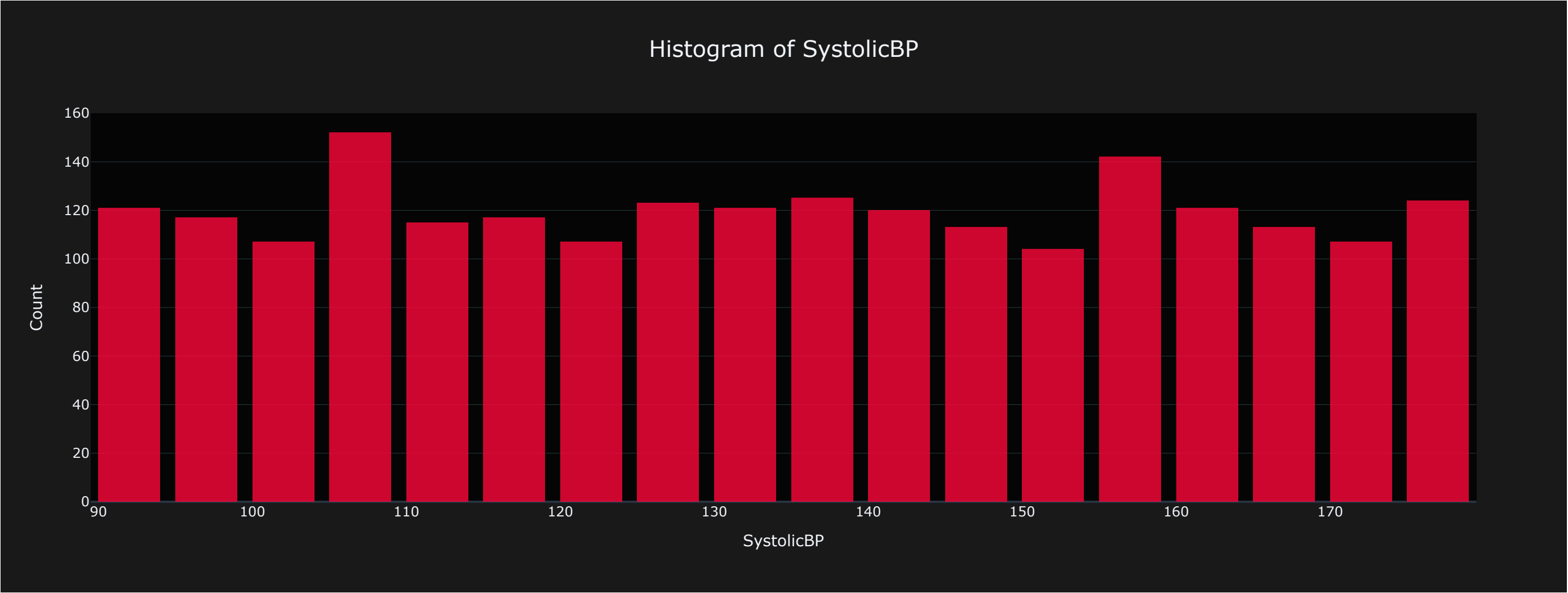
```
# Customize layout with dark theme
fig.update_layout(
    title_text=f'Histogram of {col}', # Title
    title_x=0.5, # Title position
    title_font=dict(size=20), # Title font size
    xaxis_title_text=col, # x-axis label
    yaxis_title_text='Count', # y-axis label
    bargap=0.2, # Gap between bars
    template='plotly_dark', # Dark theme
    paper_bgcolor='rgba(0,0,0,0.9)', # Dark background
```

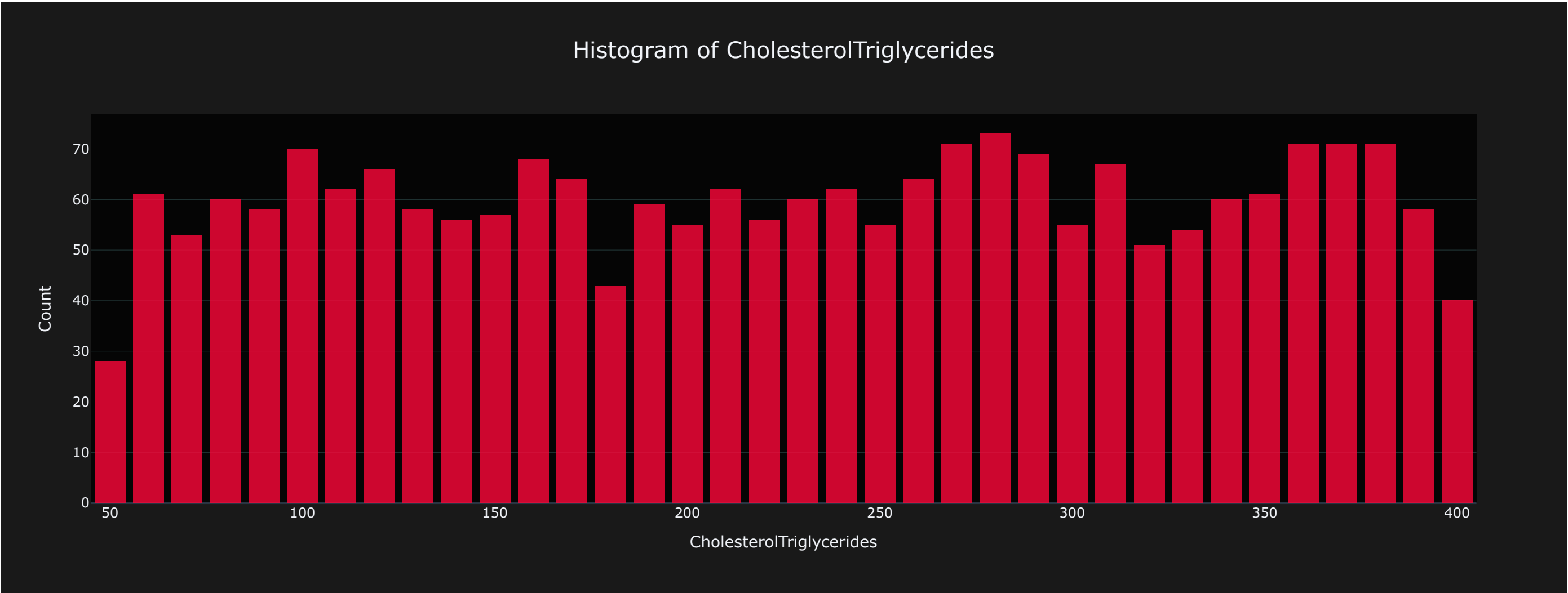
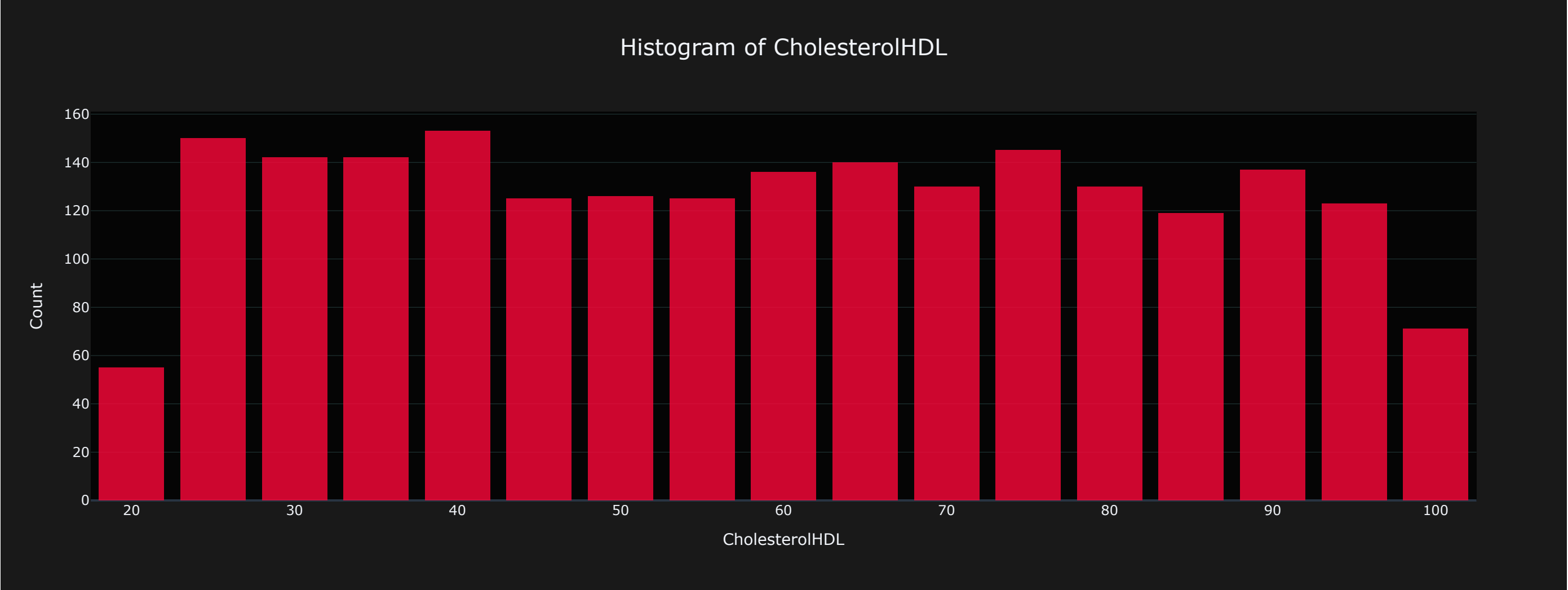
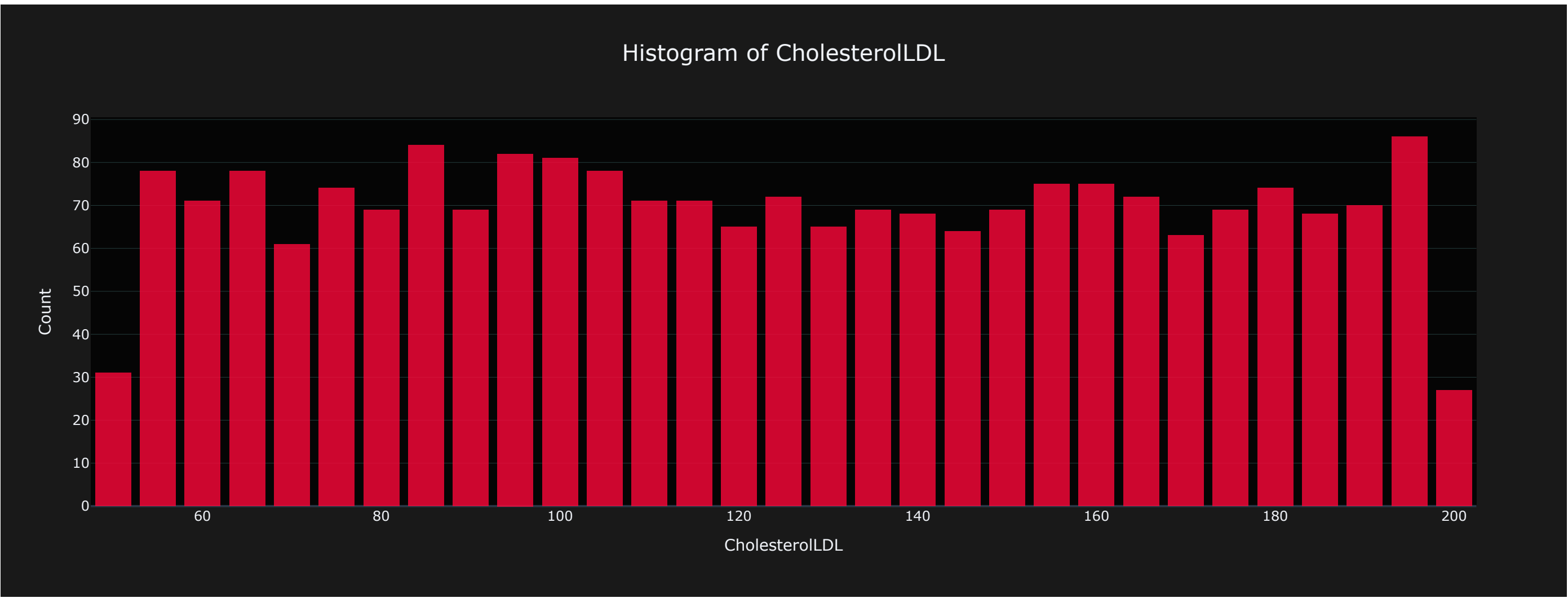


```
plot_bgcolor='rgba(0,0,0,0.8)', # Darker plot area
)

# Customize x and y axis with grid in dark mode
fig.update_xaxes(showgrid=False, zeroline=False)
fig.update_yaxes(showgrid=True, gridwidth=0.5, gridcolor='DarkSlateGray') # Darker grid color

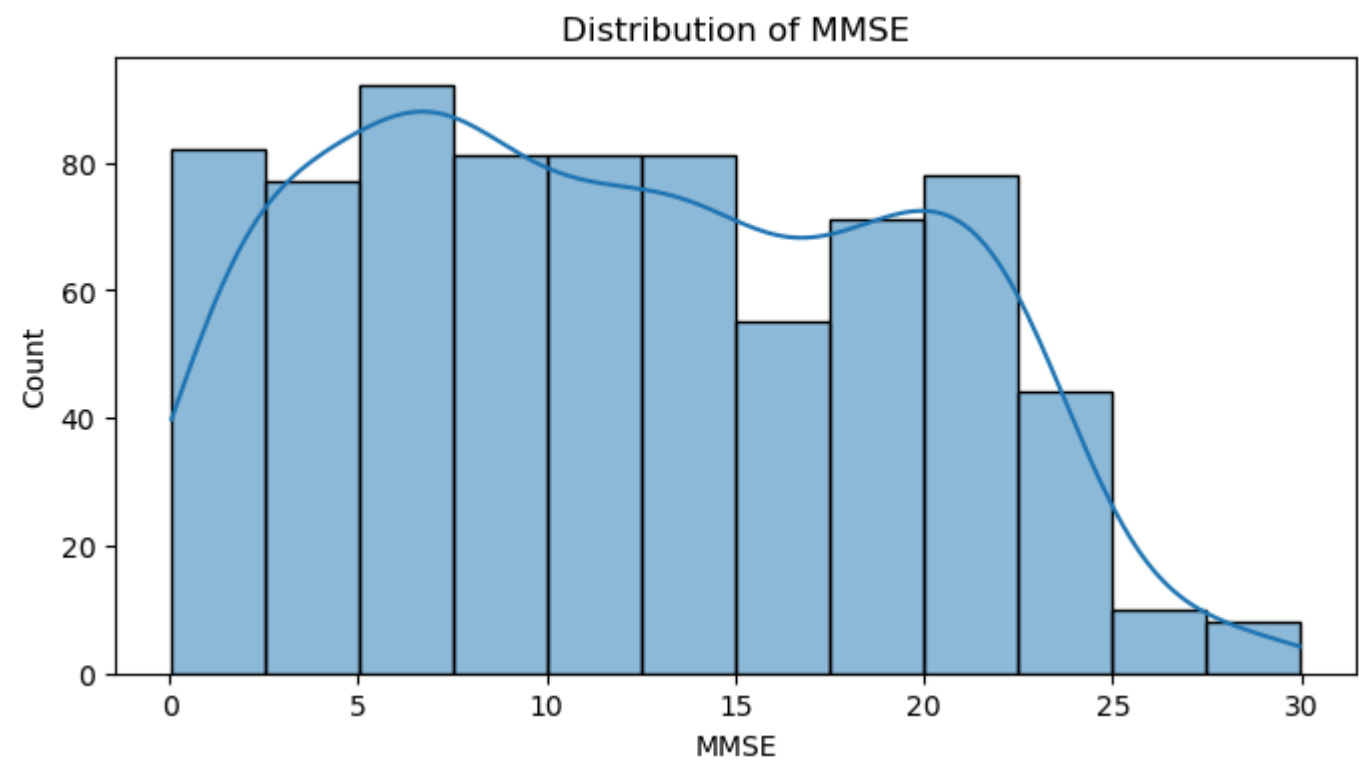
fig.show()
```





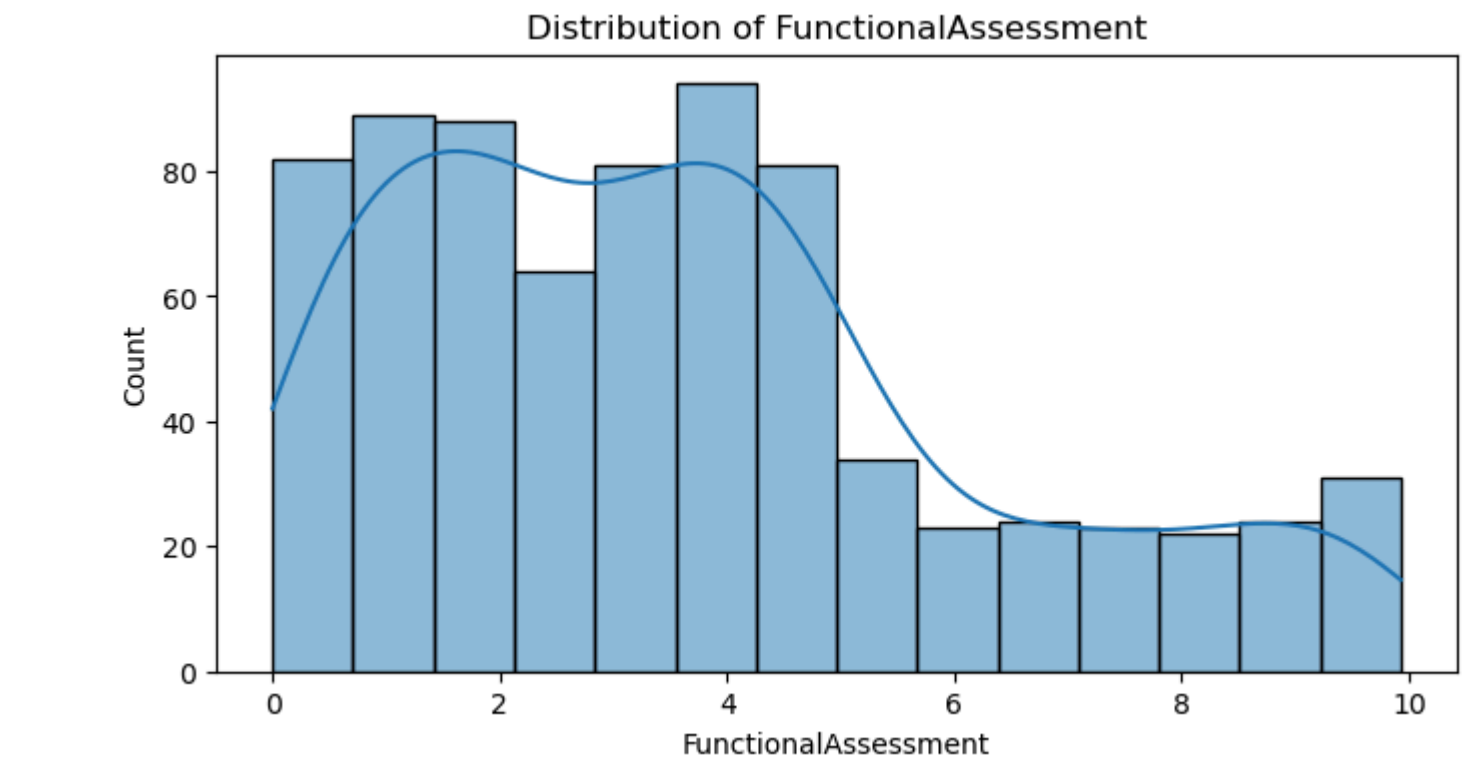
Cognitive and Functional Assessments Visualization

```
In [51]: plot("MMSE")
```



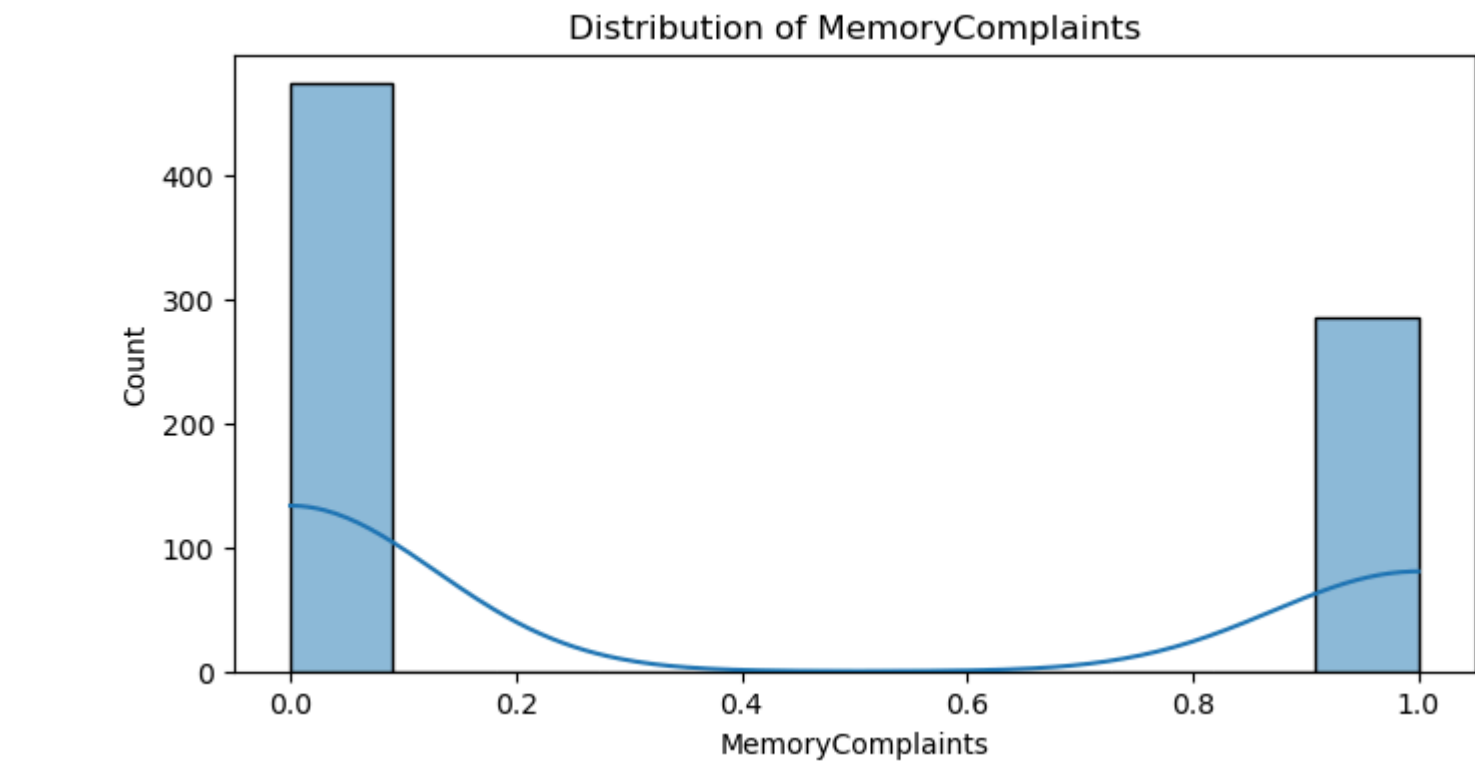
This is a skewed distribution and we can see most Diagnosed patients have lower MMSE Scores, Scores below 24 indicate some level of cognitive impairment, with lower scores showing more severe impairment, which is associated with Alzheimer's Disease, so this is an important feature.

```
In [52]: plot("FunctionalAssessment")
```

Most diagnosed patients have low scores below 4 which is bad as Lower scores indicate greater difficulty in performing daily activities, reflecting more severe functional impairment, often seen in Alzheimer's patients. This is also an Important feature

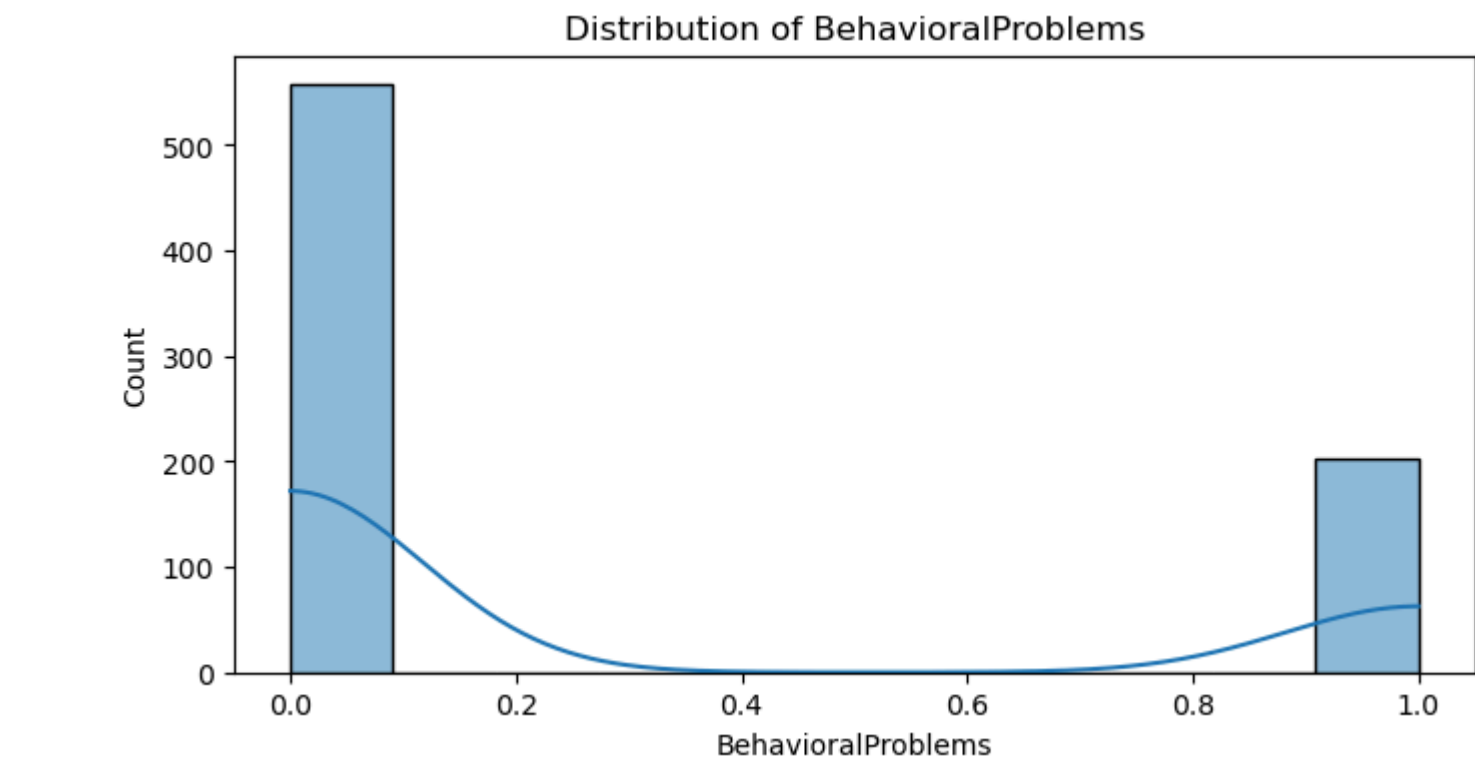
```
In [53]: plot("MemoryComplaints")
```



```
In [54]: check_percent("MemoryComplaints")
```

63.982102908277405
63-65% Patients who have Presence of memory complaints also are Diagnosed with Alzheimer's Disease

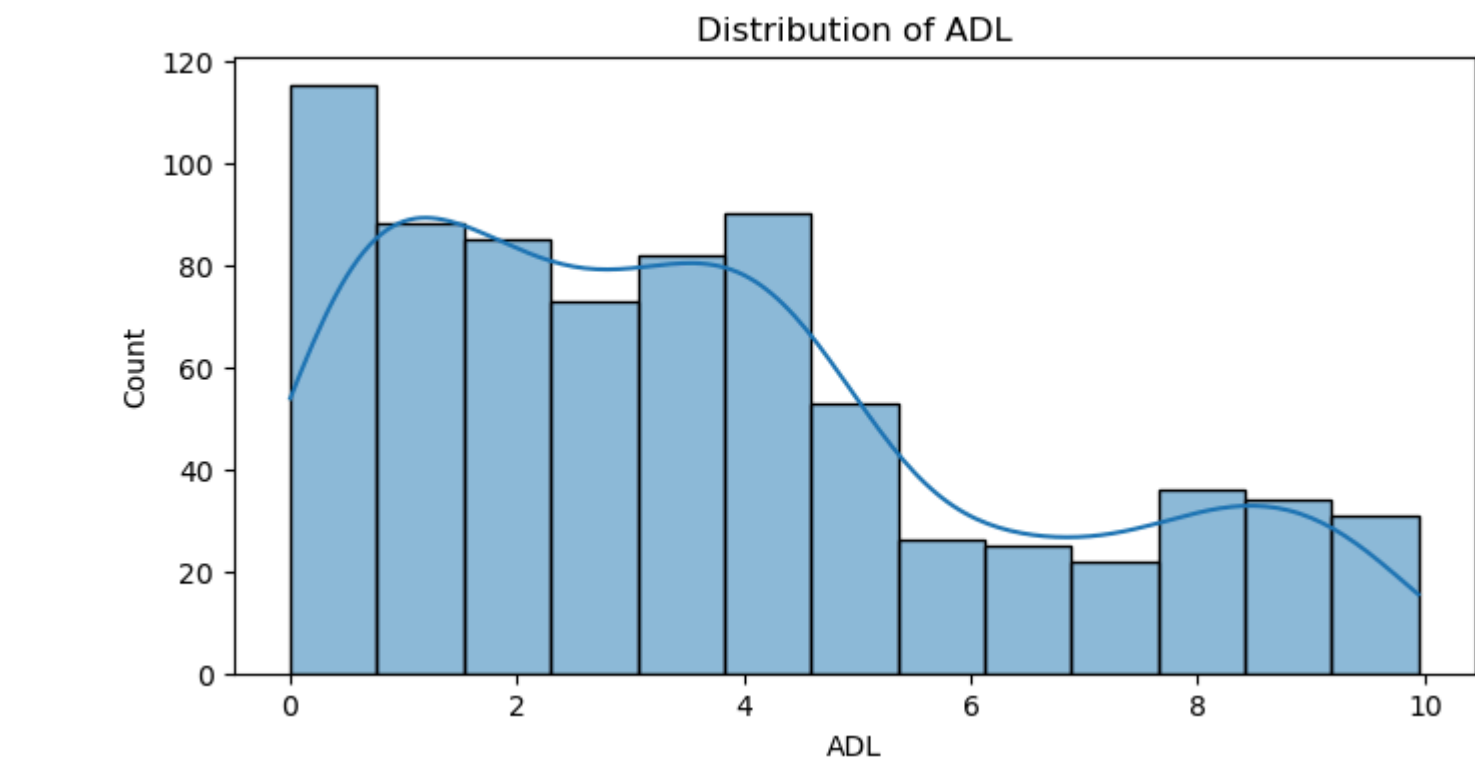
```
In [55]: plot("BehavioralProblems")
```



```
In [56]: check_percent("BehavioralProblems")
```

60.23738872403561
Patients with Behavioral Problems are 60% more likely to be Diagnosed with Alzheimer

```
In [57]: plot("ADL")
```



most patients have lower ADL scores below 4 and we know that Lower scores indicate greater impairment.

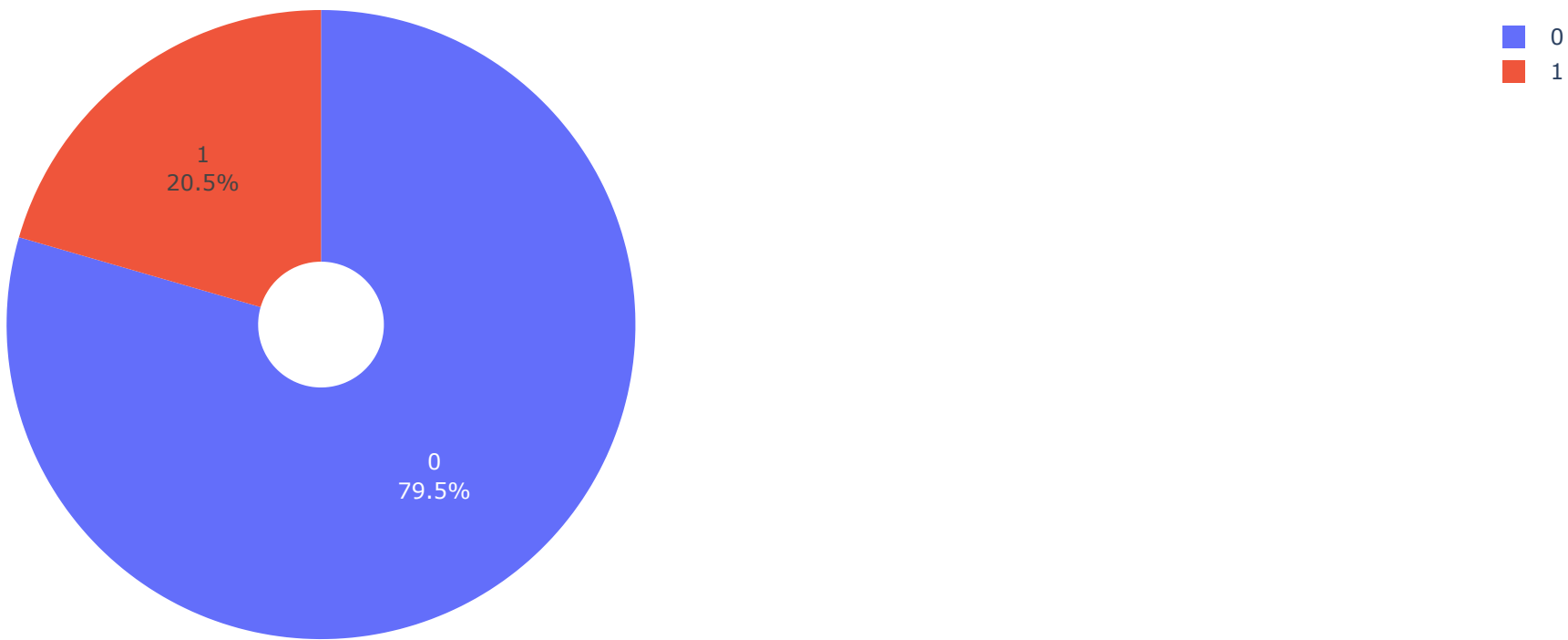
overall, the **Cognitive and Functional Assessments** columns are pretty useful and seems to contribute in determining whether a person have Alzheimer or not

Symptoms

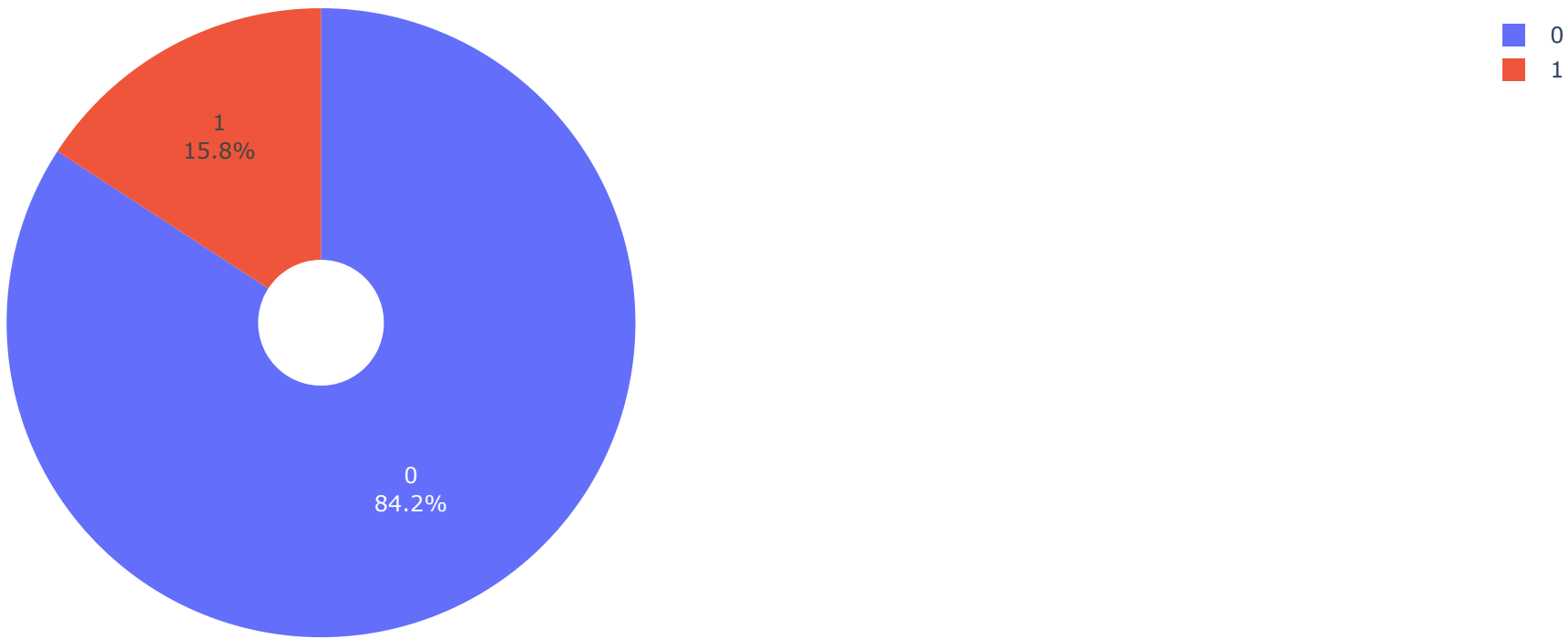
```
In [58]: symptoms = ["Confusion", "Disorientation", "PersonalityChanges", "DifficultyCompletingTasks", "Forgetfulness"]
for col in symptoms:
    fig = px.pie(values=df[col].value_counts(), names=df[col].value_counts().index, title=f"Percentage of {col}", hole=0.2)
```

```
fig.update_traces(textinfo='percent+label')
fig.show()
```

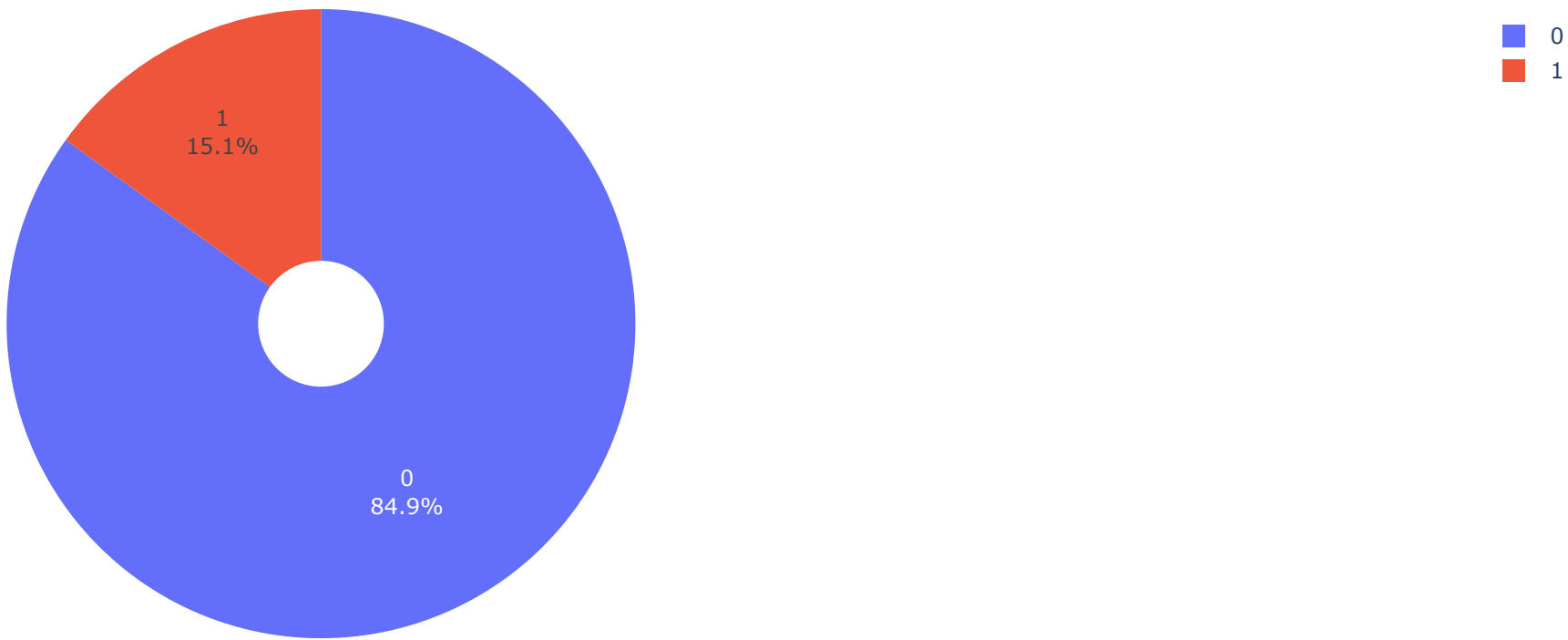
Percentage of Confusion



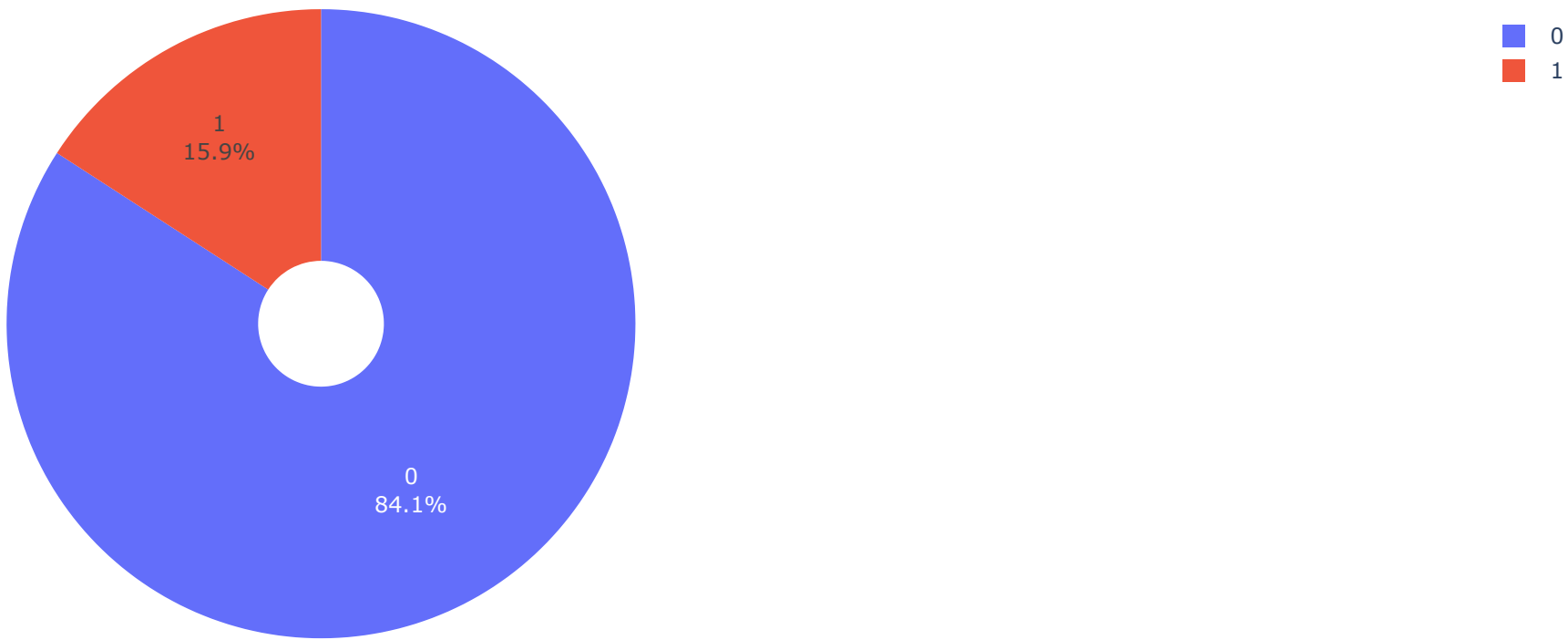
Percentage of Disorientation



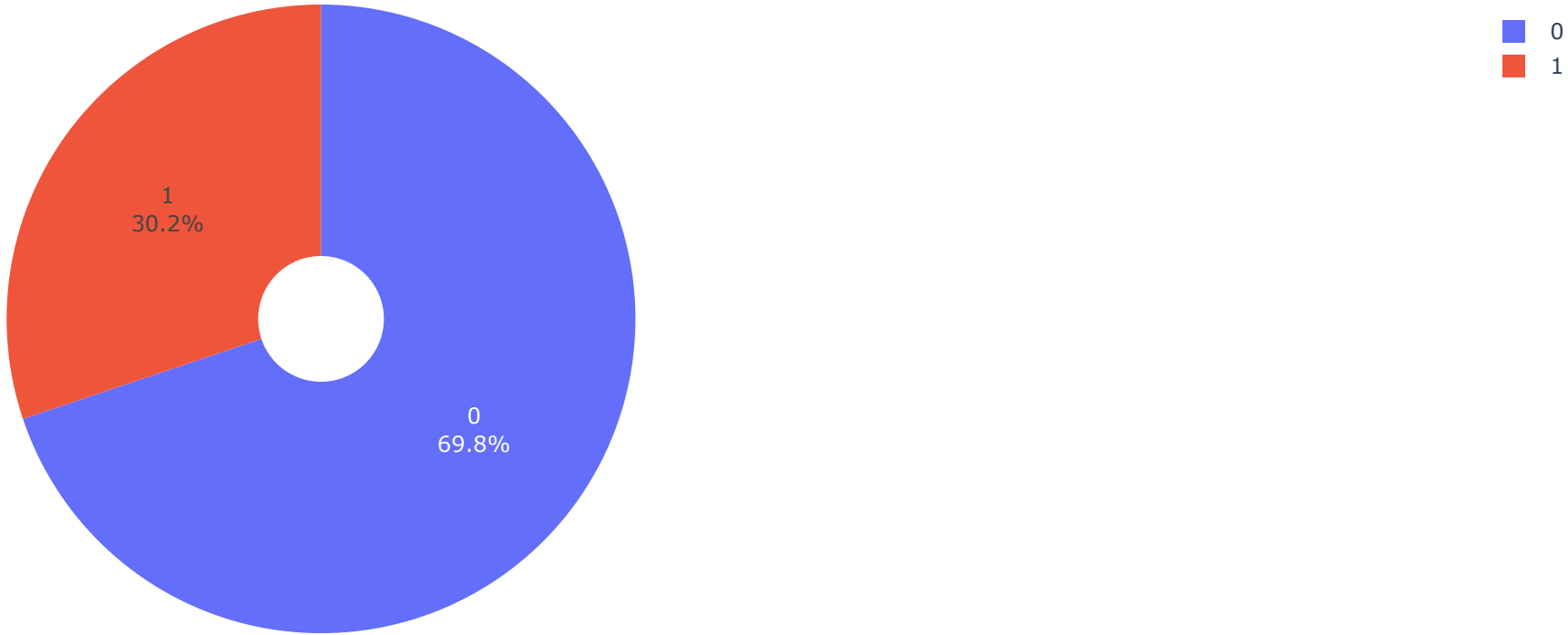
Percentage of PersonalityChanges



Percentage of DifficultyCompletingTasks



Percentage of Forgetfulness



The above features play a role in determining Alzheimer's disease, but we can see most diagnosed patients of 70-85% dont have these so it might suggest that the dataset includes patients in various stages of the disease, or that these symptoms are not uniformly reported or recorded. Early-stage Alzheimer's might present with less pronounced symptoms, or the dataset might be missing some data.

Anyways, we will see if we can do some Feature Engineering as well for these later in the Preprocessing step.

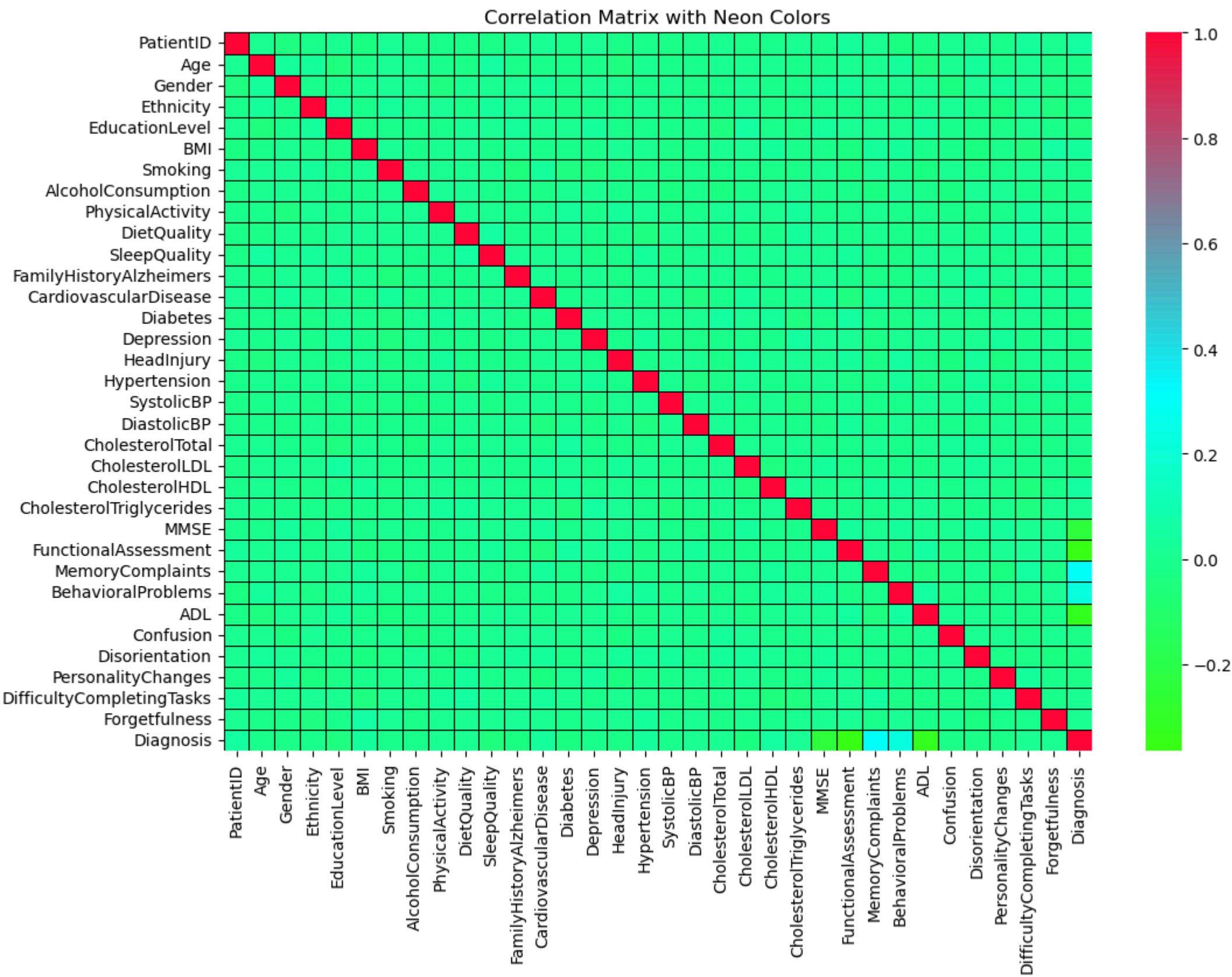
Correlation Heatmap

```
In [60]: import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.colors as mcolors

# Drop non-numeric columns
numeric_df = df.select_dtypes(include=["number"])

# Define a custom neon colormap
neon_cmap = mcolors.LinearSegmentedColormap.from_list(
    "neon", ["#39FF14", "#00FFFF", "#FF073A"], N=256 # Bright neon green, cyan, and neon red
)

# Plot the heatmap with the custom neon colormap
plt.figure(figsize=(12, 8))
sns.heatmap(numeric_df.corr(), annot=False, cmap=neon_cmap, fmt=".2f", linewidths=0.5, linecolor="black")
plt.title("Correlation Matrix with Neon Colors")
plt.show()
```



There are some strong Correlation between our Target variable with the **Cognitive and Functional Assessments** columns:

Negative correlation with MMSE, FunctionalAssesment and ADL.

Slight positive correlation with MemoryComplaints and BehavioralProblems.

Correlation with the target variable

```
In [66]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Define neon color palette
neon_colors = ["#39FF14", "#00FFFF", "#FF073A", "#FF4500", "#9400D3"]

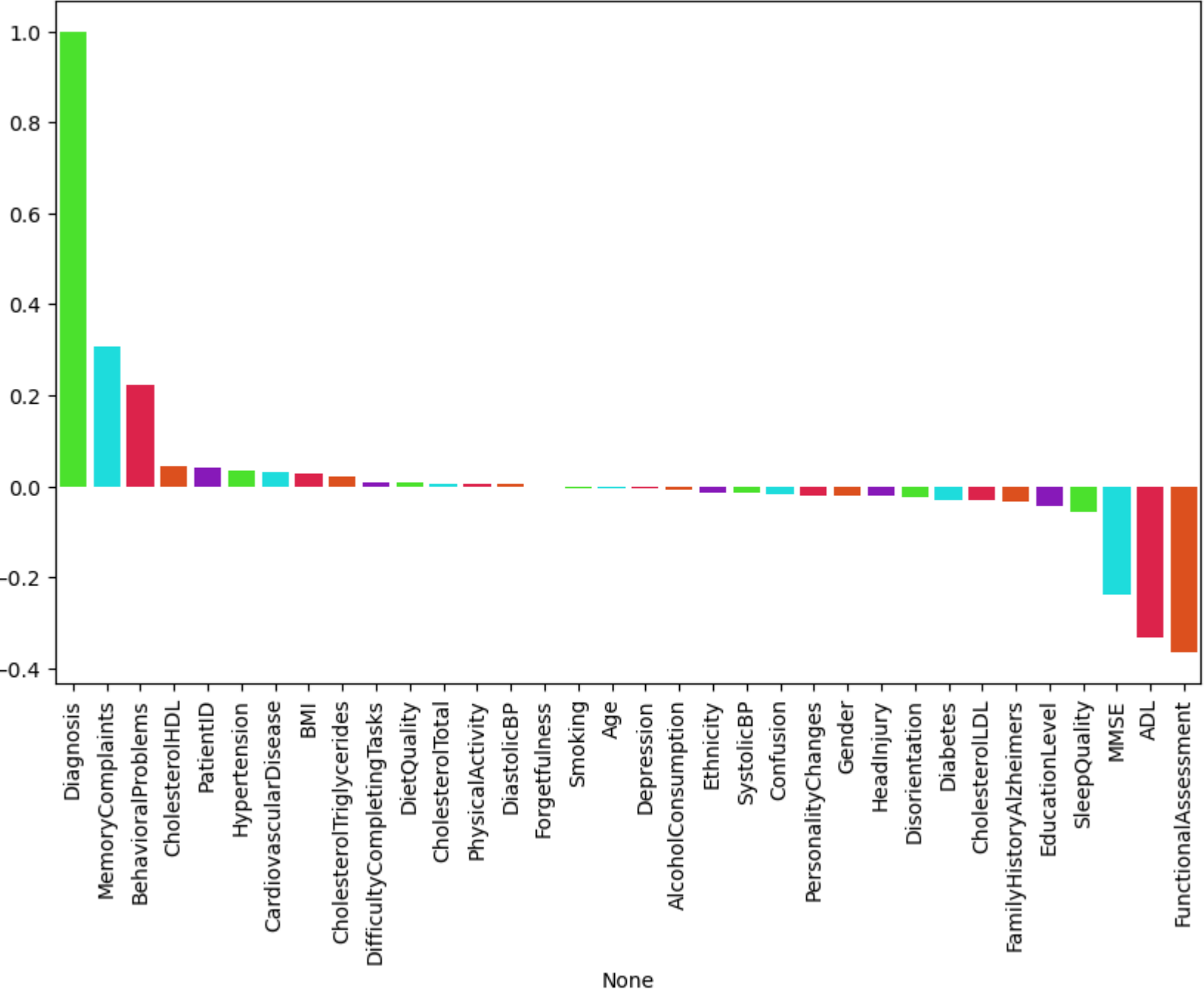
# Drop non-numeric columns
numeric_df = df.select_dtypes(include=["number"])
```



```
# Calculate the correlation of features with the 'Diagnosis' column
corr_with_target = numeric_df.corr()['Diagnosis'].sort_values(ascending=False)

# Convert the neon color palette to a List, cycling through for each bar
colors = list(np.tile(neon_colors, len(corr_with_target))[:len(corr_with_target)])

# Plotting
plt.figure(figsize=(10, 6))
sns.barplot(x=corr_with_target.index, y=corr_with_target.values, palette=colors, hue=corr_with_target.index, dodge=False, legend=False)
plt.xticks(rotation=90)
plt.title('Correlation of Features with Diagnosis', color="white")
# Set the background color to black for neon effect
plt.show()
```



CONCLUSIONS of EDA

Machine Learning

- The Dataset is pretty small with over 2k datas but 33 (Useful) Features
- No Data Cleaning is required since there is no Missing values, Duplicates, Outliers nor any Wrong formatted values
- There are no Text Categorical columns meaning no Encoding is needed either
- There are Non-Binary Numerical columns which needed to be Scaled
- There is a Moderate Imbalance in the dataset which need to be handled
- Most of the columns are more of a Uniform Distribution rather than Normal (Gaussian) Distribution
- Feature Engineering is needed for Medical History columns as they are more impactful in predicting the disease
- Cognitive and Functional Assessments columns scores are important features for predicting the disease
- See if we can do some Feature Engineering on Symptoms columns to increase their contribution in predicting the disease

Analysis Conclusions

- Alzheimer's Disease usually develops in middle or late life, with the average age of diagnosis being 70-75 years old and by age 85, the risk reaches nearly 50%.
- 75% of patients are African American.
- Adults with a BMI of 25.0–29.9 in mid-life (ages 45–55) have a higher chance of developing Alzheimer's disease or dementia later in life (ages 65 and older). Adults with a BMI of more than 30.0 in mid-life have an even higher risk of developing Alzheimer's disease
- Smoking is responsible for 14% of AD cases worldwide. Smoking can increase the risk of AD in a number of ways
- Most Patients who are diagnosed have low physical activities
- Diet quality is a critical factor in managing Alzheimer's Disease (AD) and potentially reducing the risk of developing it
- Sleep quality is a crucial aspect of managing Alzheimer's Disease (AD) and can significantly impact cognitive function, overall health, and the progression of the disease
- 32% Percent of patients who have a family history of the disease have also developed it
- Cardiovascular disease is related to Alzheimer's disease because they share several risk factors and underlying mechanisms. Poor cardiovascular health, such as high blood pressure, high cholesterol, obesity, diabetes, and smoking, can increase the risk of developing Alzheimer's. 40% of the patients with Cardiovascular Disease have developed Alzheimer's
- 30% Patients with Diabetes are diagnosed with Alzheimer
- Depression is linked to an increased risk of developing Alzheimer's disease. it is considered a significant risk factor as 34% Patients who have Depression are diagnosed with Alzheimer
- 32% Patients with Head injury have Alzheimer, People who have had a head injury are more likely to develop Alzheimer's disease. Head injuries, especially severe or repeated ones, can damage brain cells and can increase the risk of cognitive decline and dementia, including Alzheimer's disease
- Hypertension is also a significant risk factor for Alzheimer's disease with 39% patients with Hypertension are diagnosed with the disease
- 63-65% Patients who have Presence of memory complaints also are Diagnosed with Alzheimer's Disease
- Patients with Behavioral Problems are 60% more likely to be Diagnosed with Alzheimer